



Université du Québec
à Chicoutimi

Module d'informatique et de mathématique
Département d'informatique et de mathématiques

8INF950 : Sujet spécial en cybersécurité

Cybersécurité défensive : Mapping automatique de base de
données d'incident et de détection

Date : 17 août 2026

8INF950 : Sujet spécial en cybersécurité - Été 2026

Table des matières

1	Synthèse des lectures	4
1.1	Doc 1 — 99% de faux positifs	4
1.2	Doc 2 — UCO (Unified Cybersecurity Ontology)	4
2	Introduction	5
2.1	Problématique et objectifs	5
2.2	Contexte — VERIS	5
2.3	Contexte — MITRE ATT&CK	6
2.4	Contexte — Le mapping	6
2.5	Contexte — Les 7 capability groups	7
2.6	Contexte — Versions et formats de données	7
3	Méthodologie	7
3.1	La méthode actuelle de mapping	7
3.2	État de l'art — Alarmes SIEM et modèle REACT	8
3.3	État de l'art — Ontologies et unification (UCO)	8
3.4	État de l'art — Approches IA pour l'alignement sémantique	8
3.5	Notre pipeline	8
3.5.1	Architecture du projet	10
4	Solution	10
4.1	Solution RAG	10
4.1.1	Principe et objectif	10
4.1.2	Architecture et fonctionnement	11
4.1.3	Variante attack_only	12
4.1.4	Variante with_examples	14
4.1.5	Variante DeepSeek-V4-Flash-0731	17
4.1.6	Variante Llama-3.3-70B-Instruct-Turbo	18
4.1.7	Variante Qwen2.5-7B-Instruct-Turbo	19
4.1.8	Itérations analogiques Llama (v2–v14)	20
4.2	Solution Fine-tune	27
4.3	Solution PROMPT	27
4.4	Solution Fine-tune	27
4.4.1	Principe général	27
4.4.2	Données utilisées	27
4.4.3	Pipeline supervisé local	28
4.4.4	Paramètres de décision	30
4.4.5	Limites de l'approche locale	30
4.4.6	Variante LLM zero-shot	30
4.4.7	Paramétrage expérimental	31
4.4.8	Robustesse du format de sortie	32
4.4.9	Contraintes liées à l'utilisation d'une API externe	33
4.5	Solution PROMPT	34
4.5.1	Principe	34
4.5.2	Périmètre traité	34
4.5.3	Architecture logicielle	34
4.5.4	System Prompt	35

4.5.5	Modèles Together AI	35
4.5.6	Problèmes rencontrées	35
4.5.7	Correctifs effectués	35
4.5.8	Limites méthodologiques	35
4.6	Synthèse des trois solutions	36
5	Résultats	36
5.1	Configuration expérimentale	36
5.2	Résultats — Solution RAG	36
5.3	Résultats — Solution Fine-tune	36
5.3.1	Résultats du modèle supervisé local	36
5.3.2	Résultats de la variante LLM zero-shot	37
5.3.3	Test complémentaire avec Qwen	38
5.3.4	Synthèse des résultats Fine-tune	38
5.4	Résultats — Solution PROMPT	39
5.4.1	Résultat de la première run	39
5.4.2	Seconde run	39
6	Discussion	40
7	Conclusion	40

Table des figures

1	Pipeline global du projet : données brutes, trois solutions IA et comparaison aux experts.	9
2	Résultats de la variante <code>attack_only</code> sur la version cible VERIS 1.4.1 / ATT&CK 19.1.	14
3	Résultats de la variante <code>with_examples</code> sur la version cible VERIS 1.4.1 / ATT&CK 19.1.	16
4	Résultats de la variante DeepSeek-V4-Flash-0731 (<code>with_examples</code>) sur VERIS 1.4.1 / ATT&CK 19.1.	18
5	Résultats de la variante Llama-3.3-70B-Instruct-Turbo (<code>with_examples</code>) sur VERIS 1.4.1 / ATT&CK 19.1.	19
6	Résultats de la variante Qwen2.5-7B-Instruct-Turbo (<code>with_examples</code>) sur VERIS 1.4.1 / ATT&CK 19.1.	20
7	Résultats de la variante analogique v2 (Llama <code>with_examples</code>).	21
8	Résultats de la variante analogique v3 (fill des IDs experts omis).	22
9	Résultats de la variante analogique v4 (plafonds d'ajouts par scope).	22
10	Résultats de la variante analogique v5 (skip <code>social</code> / <code>value_chain</code>).	23
11	Résultats de la variante analogique v6 (fill analogique sans expansion de famille).	23
12	Résultats de la variante analogique v7 (IDs analogiques uniquement).	23
13	Résultats de la variante analogique v8 (seconde passe sur scopes sous-générés).	24
14	Résultats de la variante analogique v9 (rerank à N constant).	24
15	Résultats de la variante analogique v10 (budget analogique et score pondéré).	25
16	Résultats de la variante analogique v11 (union same-label).	25

17	Résultats de la run v11 avec appels LLM (v11_llm) : mêmes scores que le rerank analogique.	25
18	Résultats de la variante analogique v12 (union complète et LLM si analogue vide).	26
19	Résultats de la variante analogique v13 (union same-label sans LLM). . .	26
20	Résultats de la variante analogique v14 (corpus same-label, remap ATT&CK et découverte conservative).	27

1 Synthèse des lectures

1.1 Doc 1 — 99% de faux positifs

On distingue trois types de faux positifs (FP) :

- **Vraies alarmes** : ce sont les menaces réelles.
- **Fausse alarmes** : le système se déclenche sans raison légitime.
- **Alarmes bénignes** : alarmes techniquement correctes, mais correspondant à un comportement légitime.

La validation des alarmes repose encore sur un processus humain, grâce aux connaissances et à l'expérience des analystes.

Les auteurs proposent le modèle **REACT** : les alarmes doivent être

- **Reliable** (fiables) ;
- **Explained** (explicables) ;
- **Analytical** (analytiques) ;
- **Contextual** (contextuelles) ;
- **Transferable** (transférables à d'autres environnements).

Ce modèle est pensé pour guider la conception d'outils fondés sur l'apprentissage automatique.

1.2 Doc 2 — UCO (Unified Cybersecurity Ontology)

UCO est une ontologie sémantique visant à unifier et intégrer les données hétérogènes de cybersécurité issues de multiples standards. Son but est d'*améliorer la conscience situationnelle des analystes*.

Caractéristiques principales :

- Extension de l'ontologie IDS développée par le même groupe.
- Intégration des standards les plus utilisés en cybersécurité : STIX, CVE, CCE, CVSS, CAPEC, CybOX et la *Kill Chain*.
- Représentation en OWL/RDF plutôt qu'en XML, ce qui autorise le raisonnement automatique.
- 106 classes, 59 propriétés d'objet et 45 propriétés de données.
- Expressivité logique de niveau *ALCROIQ(D)*.

Apport. Inférence logique, mappée au *Linked Open Data* (DBpedia, YAGO, etc.), permettant le croisement des données cyber avec des connaissances générales.

Cas d'usage. Un prototype identifie les vulnérabilités liées à un type de logiciel à partir des données NVD. Au moyen de requêtes SPARQL, il recense les produits d'un éditeur et leurs vulnérabilités, afin de suggérer une alternative exempte de vulnérabilités ou de comparer l'impact sécuritaire d'un changement de fournisseur.

2 Introduction

2.1 Problématique et objectifs

Dans un centre d'opérations de sécurité (SOC), les analystes doivent constamment relier des incidents décrits dans un vocabulaire normalisé aux techniques adverses documentées dans des référentiels offensifs. Ce travail d'alignement sémantique permet d'enrichir la détection, contextualiser les alertes et améliorer la conscience situationnelle, mais il reste en grande partie manuel, coûteux et peu reproductible.

L'objectif de ce projet est de reproduire automatiquement, à l'aide de l'intelligence artificielle, le mapping entre le vocabulaire VERIS et le framework MITRE ATT&CK, puis de mesurer la qualité des mappings produits par rapport à la référence officielle publiée par le CTID (*Mappings Explorer*).

Nous avons choisi de mettre trois approches en concurrence :

- Un modèle **RAG** (Retrieval-Augmented Generation)
- Un modèle d'ia modifié pour être **fine-tuné** (fine-tuning supervisé)
- Un modèle d'ia généraliste avec **prompt engineering** (ingénierie de prompts).

Le but de notre contribution est de venir créer :

- Un pipeline reproductible permettant de générer les mappings entre VERIS et MITRE ATT&CK (préparation des données, génération, comparaison) ;
- Un format de sortie standardisé pour les mappings générés par nos solutions ;
- Des scripts de comparaison automatisée entre le mapping existant et les mappings générés par nos solutions ;

Le rapport s'organise comme suit :

- L'introduction présente le contexte technique (§2.2 à §2.6) ;
- La méthodologie décrit la pratique actuelle, l'état de l'art et notre cadre expérimental ;
- La section *Solution* détaille les trois approches retenues ;
- Les résultats, la discussion et la conclusion clôturent l'analyse.

2.2 Contexte — VERIS

VERIS (*Vocabulary for Event Recording and Incident Sharing*) est un vocabulaire structuré conçu pour décrire de manière homogène les incidents de cybersécurité. Son objectif principal est de fournir un cadre commun pour documenter ce qui s'est produit lors d'un incident, indépendamment de l'organisation ou de l'outil utilisé. Autrement dit, VERIS sert à normaliser la description des événements de sécurité afin de faciliter leur analyse, leur comparaison et leur partage entre analystes, équipes de réponse à incident et organismes de recherche.

Sur le plan de son organisation, VERIS repose sur une structure hiérarchique. Le vocabulaire est découpé en grands axes, comme **action**, **attribute**, **actor** ou encore **value_chain**, qui correspondent à différentes dimensions d'un incident. Chaque axe contient ensuite des catégories plus précises, par exemple **hacking**, **malware** ou **social** dans l'axe **action**, puis des sous-catégories et des valeurs plus fines telles que **variety** ou **vector**. Cette organisation permet de représenter un incident avec plusieurs niveaux de granularité, depuis une vue générale jusqu'à une description beaucoup plus détaillée du comportement observé.

VERIS a été développé à l'origine dans le cadre des travaux de Verizon autour du *Data Breach Investigations Report* (DBIR), avec l'idée de disposer d'un schéma cohérent pour collecter et comparer les données d'incidents à grande échelle. Avec le temps, il est devenu un standard largement reconnu dans le domaine de la réponse à incident et de l'analyse cyber, notamment parce qu'il permet d'agréger des observations provenant de sources variées tout en conservant une structure commune. Dans le cadre de ce projet, VERIS joue donc le rôle de référentiel décrivant le “quoi” de l'incident, que l'on cherche ensuite à relier au “comment” représenté par les techniques de MITRE ATT&CK.

2.3 Contexte — MITRE ATT&CK

MITRE ATT&CK (*Adversarial Tactics, Techniques, and Common Knowledge*) est une base de connaissances dédiée à la description des comportements adverses observés lors d'intrusions réelles. Contrairement à un simple catalogue de vulnérabilités ou d'outils, ATT&CK cherche à modéliser la manière dont un attaquant agit au cours d'une opération, c'est-à-dire les étapes, les objectifs intermédiaires et les techniques mobilisées. Dans cette perspective, ATT&CK décrit le “comment” d'une attaque, là où d'autres référentiels, comme VERIS, décrivent davantage le “quoi” de l'incident.

Le framework est organisé de manière hiérarchique autour de **tactiques** et de **techniques**. Les tactiques représentent les grands objectifs poursuivis par l'attaquant, par exemple l'accès initial, l'exécution, la persistance, l'élévation de privilèges ou l'impact. À l'intérieur de chaque tactique, ATT&CK recense des techniques identifiées par un code de type T1059, accompagnées d'un nom, d'une description et d'un ensemble de tactiques associées. Certaines techniques sont elles-mêmes décomposées en **sous-techniques**, notées par exemple T1059.001, ce qui permet de représenter plus finement des variantes de comportement ou des modes opératoires particuliers.

MITRE ATT&CK est maintenu par la MITRE Corporation à partir de retours d'expérience opérationnels, de rapports d'incidents et d'observations issues du renseignement sur les menaces. Il s'est progressivement imposé comme une référence majeure en cybersécurité défensive, notamment pour la détection, la chasse aux menaces, l'évaluation de couverture et la structuration des connaissances sur les attaques. Dans le cadre de ce projet, ATT&CK constitue le référentiel cible vers lequel les capacités VERIS sont mappées, afin de relier la description d'un incident aux techniques adverses susceptibles d'en expliquer le déroulement.

2.4 Contexte — Le mapping

Le mapping étudié dans ce projet consiste à relier des capacités issues du vocabulaire VERIS, qui décrivent la nature et les caractéristiques d'un incident, à des techniques du framework MITRE ATT&CK, qui décrivent les comportements et modes opératoires adverses susceptibles d'expliquer cet incident. Autrement dit, il s'agit de faire le lien entre le “quoi” observé dans VERIS et le “comment” modélisé dans ATT&CK. Concrètement, une capacité VERIS, par exemple `action.hacking.variety.SQLi`, peut être associée à une ou plusieurs techniques ATT&CK jugées pertinentes par les experts.

Ces correspondances sont produites et publiées par le CTID / MITRE Engenuity. Elles constituent dans notre travail une double référence : d'une part, elles définissent la cible que nos solutions d'intelligence artificielle cherchent à reproduire automatiquement ; d'autre part, elles servent de base d'évaluation pour mesurer la qualité des mappings

générés. Le problème traité dans ce projet n'est donc pas seulement de produire des associations plausibles entre VERIS et ATT&CK, mais bien de se rapprocher le plus possible d'un mapping expert déjà établi.

2.5 Contexte — Les 7 capability groups

Sept groupes VERIS dits « comportementaux » sont mappés vers ATT&CK. Le Tableau 1 présente les volumes pour la paire de référence VERIS 1.4.1 / ATT&CK 19.1.

Capability group	Paires VERIS↔ATT&CK	Capacités VERIS
action.hacking	538	68
action.malware	405	54
action.social	78	23
attribute.integrity	91	13
attribute.confidentiality	73	1
attribute.availability	44	6
value_chain.development	25	12

TABLEAU 1 – Les sept capability groups VERIS mappés (référence 1.4.1 / 19.1).

Cas particulier : pour `attribute.confidentiality`, les experts mappent le champ `data_disclosure` lui-même (une seule capacité), et non des valeurs sous `variety`.

2.6 Contexte — Versions et formats de données

Quatre paires de versions sont supportées de bout en bout : ATT&CK 9.0 / VERIS 1.3.5, 12.1 / 1.3.7, 16.1 / 1.4.0 et 19.1 / 1.4.1.

La **version cible d'évaluation** de ce rapport est VERIS 1.4.1 / ATT&CK 19.1 (clé `veris-1.4.1_attack-19.1-enterprise`).

Les entrées préparées pour chaque solution sont :

- `veris_<v>.json` : capacités VERIS dans les sept groupes (les « questions ») ;
- `attack_<a>.json` : techniques ATT&CK non dépréciées (les « candidats »).

Les sorties des solutions suivent le format `veris_to_mitre` ; la référence experte utilise le format CTID (`mapping_objects`).

3 Méthodologie

3.1 La méthode actuelle de mapping

Aujourd'hui, le mapping VERIS → ATT&CK est réalisé par des experts du CTID / MITRE Engenuity [?]. Le processus est essentiellement **manuel** : pour chaque capacité VERIS, l'expert analyse sa sémantique, consulte le catalogue ATT&CK et sélectionne les techniques correspondantes (y compris les sous-techniques lorsque nécessaire). Les résultats sont publiés via l'outil *Mappings Explorer* au format CTID.

Cette approche présente des **avantages** indiscutables : haute qualité, validation par des spécialistes et référence largement adoptée. Ses **limites** sont toutefois significatives : coût en temps humain, faible scalabilité et nécessité de recommencer le travail à chaque

nouvelle version de VERIS ou d'ATT&CK. Notre projet vise précisément à automatiser cette tâche tout en mesurant l'écart par rapport à cette référence experte.

3.2 État de l'art — Alarmes SIEM et modèle REACT

On distingue trois types de faux positifs (FP) dans les systèmes SIEM [?] :

- **Vraies alarmes** : menaces réelles.
- **Fausse alarmes** : déclenchement sans raison légitime.
- **Alarmes bénignes** : techniquement correctes, mais correspondant à un comportement légitime.

La validation des alarmes repose encore largement sur l'expertise humaine. Les auteurs proposent le modèle **REACT** : les alarmes doivent être **R**eliable (fiables), **E**xplained (explicables), **A**nalytical (analytiques), **C**ontextual (contextuelles) et **T**ransferable (transférables). Ce cadre guide la conception d'outils fondés sur l'apprentissage automatique. Dans notre projet, l'automatisation du mapping VERIS $\leftrightarrow$ ATT&CK vise à réduire la charge cognitive des analystes en normalisant plus rapidement le lien entre incidents et techniques adverses.

3.3 État de l'art — Ontologies et unification (UCO)

UCO (*Unified Cybersecurity Ontology*) [?] est une ontologie sémantique visant à unifier des données hétérogènes de cybersécurité (STIX, CVE, CAPEC, CybOX, *Kill Chain*, etc.) en OWL/RDF, ce qui autorise le raisonnement automatique. L'alignement VERIS $\leftrightarrow$ ATT&CK s'inscrit dans cette problématique plus large d'**unification de taxonomies cyber** : deux vocabulaires distincts doivent être reliés de manière cohérente pour améliorer la conscience situationnelle des analystes.

3.4 État de l'art — Approches IA pour l'alignement sémantique

Trois familles d'approches sont comparées dans ce projet :

- **Prompt engineering** : un LLM généraliste produit le mapping à partir d'instructions (zero-shot ou few-shot).
- **Fine-tuning** : un modèle est entraîné sur des paires expert annotées. Spécialisation forte et nous supposons une faible hallucination, mais avec un coût d'entraînement plus élevé.
- **RAG** [?] : un contexte pertinent (techniques ATT&CK, exemples experts) est récupéré avant la décision. Compromis entre interprétabilité, coût et performance.

3.5 Notre pipeline

Notre pipeline expérimental s'organise en quatre étapes principales, illustrées à la Figure 1. L'objectif est de partir de sources officielles, de produire des mappings automatisés VERIS $\rightarrow$ ATT&CK à l'aide de trois approches IA, puis de les comparer au mapping expert CTID.

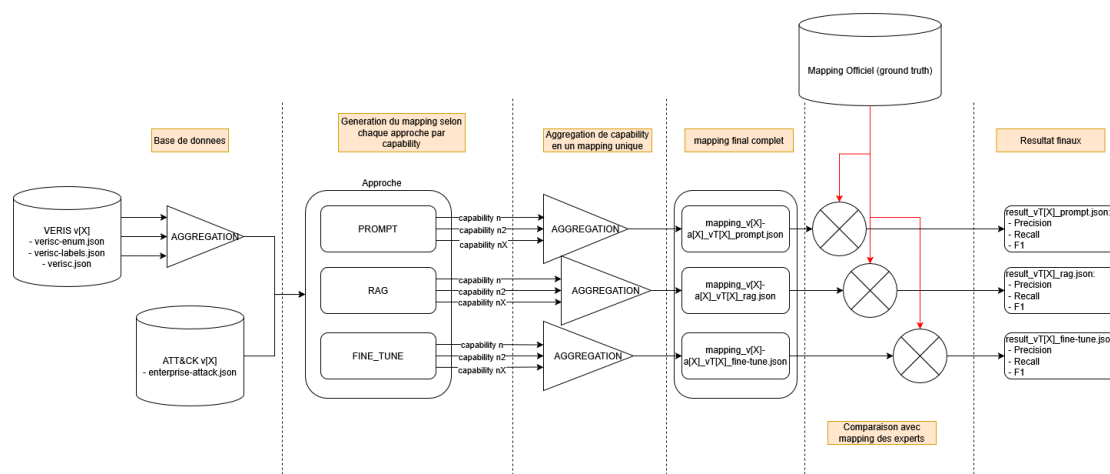


FIGURE 1 – Pipeline global du projet : données brutes, trois solutions IA et comparaison aux experts.

1. **Téléchargement des données brutes.** Les sources officielles sont rassemblées dans le dossier `data/raw/`. On y trouve les fichiers VERIS (`verisc-enum.json`, `verisc-labels.json`, `verisc.json`), le catalogue ATT&CK au format STIX (`enterprise-attack.json`) ainsi que les mappings experts publiés par le CTID (CSV, YAML, JSON ou STIX selon la version). Quatre paires de versions sont disponibles, de VERIS 1.3.5 / ATT&CK 9.0 à VERIS 1.4.1 / ATT&CK 19.1.
2. **Préparation des entrées.** Le script `tools/build_work_inputs.py` transforme ces données brutes en fichiers exploitables par les solutions IA, stockés dans `data_for_work/`. Pour chaque paire de versions, il produit :
 - `veris_<v>.json` : les capacités VERIS des sept capability groups (les « questions » à mapper) ;
 - `attack_<a>.json` : les techniques ATT&CK non dépréciées (les « candidats ») ;
 - `mapping_des_experts.json` : une copie locale du mapping de référence, utilisée pour la comparaison.
3. **Génération des mappings.** Chacune des trois solutions (`Solution_RAG`, `Solution_PROMPT`, `Solution_FINE_TUNE`) ingère les fichiers préparés et produit **sept fichiers JSON** au format `veris_to_mitre`, soit un fichier par capability group (`action.hacking`, `action.malware`, `action.social`, `attribute.integrity`, `attribute.confidentiality`, `attribute.availability`, `value_chain.development`). Les résultats sont écrits dans `Resultat/Resultat_<APPROCHE>/`.
4. **Comparaison aux experts.** Le script `Resultat/compare_veris_mappings_v2.py` confronte automatiquement les sorties des solutions au mapping expert CTID (`Resultat/Mapping_des_experts/`). Il valide la présence des sept fichiers, calcule les métriques de recouvrement (précision, rappel, F1, Jaccard) sur les paires (`veris_id`, `attack_id`) et classe les solutions entre elles.

Principe anti-fuite. Pour la version cible d'évaluation (VERIS 1.4.1 / ATT&CK 19.1), le mapping expert correspondant n'est **jamais** utilisé comme exemple d'entraînement ou d'indexation dans les solutions IA. Seules les versions antérieures (9.0, 12.1, 16.1) peuvent servir d'exemples analogiques, notamment dans la variante RAG `with_examples`.

3.5.1 Architecture du projet

Le dépôt SIEM est organisé de manière modulaire afin de séparer clairement les données, leur préparation, l'implémentation des solutions IA et l'évaluation des résultats. Cette organisation facilite la reproductibilité du pipeline et permet de traiter plusieurs paires de versions VERIS / ATT&CK dans un même cadre expérimental.

- **data/** : contient les **sources brutes** issues des dépôts officiels. Le sous-dossier **raw/attack/** regroupe les catalogues ATT&CK au format STIX, **raw/veris/** les schémas et énumérations VERIS, et **raw/mapping/** les mappings experts publiés par le CTID. Le dossier **databases/** regroupe également des bases SQLite auxiliaires utilisées pour certains vocabulaires et correspondances.
- **data_for_work/** : regroupe les **entrées préparées** pour chaque paire de versions, dans des sous-dossiers nommés selon la convention **attack-<a>_veris-<v>**. Chaque dossier contient les fichiers **veris-<v>.json** (capacités VERIS à mapper), **attack-<a>.json** (techniques ATT&CK candidates) et **mapping_des_experts.json** (copie locale de la référence experte).
- **tools/** : regroupe les **scripts de préparation et de validation** du pipeline. Le script **build_work_inputs.py** transforme les données brutes en fichiers exploitables dans **data_for_work/**, tandis que **build_example_results.py** génère des sorties de référence au format **veris_to_mitre**, utiles pour valider le format attendu.
- **Solution/** : contient le **code des trois approches IA** mises en concurrence : **Solution_RAG**, **Solution_PROMPT** et **Solution_FINE_TUNE**. Chaque solution lit les mêmes entrées préparées et produit sept fichiers JSON, un par capability group VERIS.
- **Resultat/** : centralise les **sorties expérimentales** et les outils de comparaison. On y trouve les mappings experts officiels dans **Mapping_des_experts/**, les résultats produits par chaque approche dans **Resultat_RAG/**, **Resultat_PROMPT/** et **Resultat_FINE_TUNE/**, ainsi que les scripts **compare_veris_mappings.py** et **compare_veris_mappings_v2.py** utilisés pour mesurer la qualité des mappings générés.

Cette architecture matérialise le flux décrit précédemment : les données brutes sont d'abord normalisées, puis transformées en entrées de travail, ensuite traitées par les solutions IA, et enfin comparées au mapping expert CTID.

4 Solution

Les trois solutions partagent les mêmes entrées (**veris-<v>.json**, **attack-<a>.json**) et la même sortie attendue (sept fichiers **veris_to_mitre**), mais diffèrent par leur stratégie de décision IA.

4.1 Solution RAG

4.1.1 Principe et objectif

La solution **RAG** (*Retrieval-Augmented Generation*) [?] vise à reproduire automatiquement le mapping expert entre les capacités VERIS et les techniques MITRE ATT&CK. Contrairement à une approche purement basée sur un prompt ou sur un modèle fine-tuné,

le RAG **ancrer chaque décision dans un contexte récupéré** à partir de connaissances indexées localement.

L'objectif est double :

- **réduire le risque d'hallucination**, en forçant la solution à s'appuyer sur des techniques ATT&CK réellement présentes dans le catalogue cible, plutôt que sur des identifiants inventés ;
- **exploiter une similarité sémantique** entre la description d'une capacité VERIS et les descriptions des techniques adverses.

Concrètement, pour chaque capacité VERIS, la solution construit une requête sémantique, récupère les éléments les plus pertinents dans une base vectorielle, puis produit une proposition de mapping au format `veris_to_mitre`. Dans notre expérimentation principale, le backend utilisé est `retrieval` : la décision repose sur la similarité cosinus, sans appel à un LLM. Les backends `local_llm` reste néanmoins disponible pour filtrer ou raffiner les candidats récupérés.

À partir de cette architecture commune, deux variantes sont ensuite étudiées : `attack_only` et `with_examples`. Elles partagent le même pipeline global et ne diffèrent que par le contexte injecté au moment du retrieval.

4.1.2 Architecture et fonctionnement

Cette sous-section décrit la **solution RAG globale**, c'est-à-dire le pipeline commun à toute les variantes. Les améliorations propres à chaque variante sont détaillées ensuite.

Composants communs. La solution repose sur une architecture modulaire : chargement des données VERIS et ATT&CK, calcul d'embeddings, stockage vectoriel dans ChromaDB, récupération des candidats, décision de mapping, puis validation et écriture des résultats. Les principaux modules logiciels sont présentés dans le Tableau 2.

Module	Rôle
<code>config.py</code>	Versions cible, chemins et hyperparamètres
<code>datasets.py</code>	Chargement VERIS, ATT&CK et exemples experts
<code>embeddings.py</code>	Calcul des embeddings (sentence-transformers)
<code>vectorstore.py</code>	Gestion des collections ChromaDB
<code>ingest.py</code>	Indexation des connaissances
<code>retrieve.py</code>	Récupération des candidats pertinents
<code>prompt.py</code>	Construction des prompts et du schéma JSON
<code>generator.py</code>	Backends de sélection / génération
<code>generate_mapping.py</code>	Orchestration du pipeline complet

TABLEAU 2 – Modules principaux de la solution RAG.

Indexation et base vectorielle. Dans tous les cas, le catalogue ATT&CK cible est indexé dans la collection `attack_techniques`. Chaque technique est représentée par un embedding construit à partir de son identifiant, de son nom et de sa description. La similarité utilisée est la similarité cosinus. Selon la variante, une seconde collection peut également être exploitée pour enrichir le contexte ; ce point est présenté dans les sous-sections suivantes.

Flux global de traitement. Le traitement d'une capacité VERIS suit toujours la même chaîne :

1. construction d'une requête sémantique à partir du capability group, du libellé et de la description ;
2. récupération des **top- k** techniques ATT&CK les plus similaires dans **attack_techniques** ;
3. sélection ou génération des mappings, selon le backend configuré (**retrieval** ou **local_llm**) ;
4. validation anti-hallucination : rejet des identifiants ATT&CK absents du catalogue local ;
5. enrichissement des noms, sous-techniques et tactiques à partir du catalogue ATT&CK ;
6. agrégation des entrées au format **veris_to_mitre**, regroupées en sept fichiers JSON, un par capability group.

Paramètres communs. Le paramètre principal du retrieval est **RAG_TOP_K_TECHNIQUES** (nombre de techniques candidates récupérées, 20 par défaut). Les deux variantes expérimentées partagent donc la même base technique, la même validation et le même format de sortie ; seule la nature du contexte récupéré change. C'est précisément cette différence qui est évaluée dans les sous-sections **attack_only** et **with_examples**.

Backend de décision. Deux backends de décision sont disponibles après l'étape de retrieval. Dans notre expérimentation, nous utilisons exclusivement le backend **retrieval**, qui n'appelle **aucun LLM** : les techniques candidates sont filtrées selon leur similarité cosinus avec la capacité VERIS. Concrètement, une similarité supérieure ou égale à 0,50 est considérée comme **high**, une similarité supérieure ou égale à 0,38 comme **medium** ; en dessous de ce seuil, le candidat est rejeté. Le backend **local_llm** permettrait de confier au modèle local le choix final parmi les candidats récupérés, mais il n'a pas été utilisé pour les résultats présentés dans ce rapport.

Backend de décision : Calcul de la similarité cosinus. Concrètement, chaque texte — capacité VERIS ou technique ATT&CK — est d'abord transformé en vecteur numérique par le modèle d'embeddings **sentence-transformers** (**all-MiniLM-L6-v2** en mode local), avec une normalisation de norme unitaire. La proximité entre une requête VERIS **q** et une technique ATT&CK **d** est alors mesurée par la similarité cosinus $\cos(\mathbf{q}, \mathbf{d}) = \mathbf{q} \cdot \mathbf{d} / (\|\mathbf{q}\| \|\mathbf{d}\|)$, qui se réduit au produit scalaire lorsque les vecteurs sont normalisés. ChromaDB stocke ces vecteurs dans un espace cosinus et renvoie une *distance* définie par $1 - \cos(\mathbf{q}, \mathbf{d})$; notre backend **retrieval** reconvertit ensuite cette distance en similarité ($\text{sim} = 1 - \text{distance}$) afin d'appliquer les seuils de sélection (0,50 pour **high**, 0,38 pour **medium**). Ainsi, plus deux descriptions sont sémantiquement proches, plus leur similarité cosinus est élevée, et plus la technique a de chances d'être retenue dans le mapping.

4.1.3 Variante **attack_only**

Principe et corpus. La variante **attack_only** constitue la configuration de base du RAG. Son principe est simple : pour chaque capacité VERIS, le système ne s'appuie que sur le catalogue ATT&CK cible afin de proposer des techniques correspondantes. Aucun mapping expert issu de versions antérieures n'est injecté dans le contexte.

Le corpus utilisé se limite donc à la collection vectorielle `attack_techniques`, qui contient les techniques et sous-techniques ATT&CK de la version évaluée (19.1). Chaque document indexé regroupe typiquement l'identifiant, le nom et la description de la technique. L'idée est de mesurer la qualité d'un mapping fondé uniquement sur la similarité sémantique entre le vocabulaire d'incident VERIS et le référentiel adverse ATT&CK.

Fonctionnement. Le fonctionnement reprend le pipeline global décrit précédemment, avec une restriction sur l'étape de retrieval :

1. la capacité VERIS est transformée en requête sémantique (capability group, libellé, description) ;
2. les `top-k` techniques ATT&CK les plus proches sont récupérées dans `attack_techniques` ;
3. le backend de décision retient les mappings proposés : en mode `retrieval`, aucun LLM n'est appelé ; les candidats sont filtrés uniquement selon des seuils de similarité sémantique ;
4. les identifiants hors catalogue sont rejetés, puis les libellés et tactiques sont enrichis depuis le catalogue local ;
5. le résultat est écrit au format `veris_to_mitre` dans le dossier `Resultat/Resultat_RAG/..._RAG`

Cette variante sert de **baseline** : elle permet d'évaluer ce que le RAG peut obtenir sans aide supplémentaire, avant de mesurer l'apport des exemples experts dans la variante suivante.

Résultats et remarques. Voici les résultats obtenus par la variante `attack_only` sur la version cible en sorti de notre pipeline expérimental.

```
Solution : Resultat_RAG
Dossier : veris-1.4.1_attack-19.1-enterprise_RAG_attack_only
Version : veris-1.4.1_attack-19.1-enterprise
Experts : veris-1.4.1_attack-19.1-enterprise_json.json
Fichiers : 7/7 scopes presents

GLOBAL (tous scopes agregés) :
  Paires experts=1250 solution=2679 communes= 193
  Precision= 7.2% Rappel= 15.4% F1= 9.8% Jaccard= 5.2%

PAR CAPABILITY_GROUP :
```

scope	exp	sol	comm	Prec	Rapp	F1
action.hacking	534	1061	72	6.8%	13.5%	9.0%
action.malware	405	1056	61	5.8%	15.1%	8.4%
action.social	79	230	26	11.3%	32.9%	16.8%
attribute.integrity	91	151	15	9.9%	16.5%	12.4%
attribute.confidentiality	73	4	1	25.0%	1.4%	2.6%
attribute.availability	44	25	3	12.0%	6.8%	8.7%
value_chain.development	25	152	15	9.9%	60.0%	16.9%

FIGURE 2 – Résultats de la variante `attack_only` sur la version cible VERIS 1.4.1 / ATT&CK 19.1.

La Figure 2 présente la sortie de `compare_veris_mappings_v2.py` pour la variante `attack_only`. Au niveau global, sur les 1 250 paires expertes de référence, la solution en propose 2 679, dont seulement 193 sont correctes. Cela se traduit par une précision de 7,2 %, un rappel de 15,4 % et un F1 de 9,8 % (Jaccard 5,2 %). Autrement dit, le système sur-génère largement des mappings : il produit plus du double de paires par rapport aux experts, mais n'en retrouve qu'une faible partie.

Le détail par capability group confirme cette tendance. Les deux scopes les plus volumineux, `action.hacking` et `action.malware`, restent proches de la moyenne globale (F1 de 9,0 % et 8,4 %), avec là aussi beaucoup plus de propositions que de paires expertes. Les meilleurs F1 s'observent sur `action.social` (16,8 %) et `value_chain.development` (16,9 %), ce dernier bénéficiant d'un rappel élevé (60 %) au prix d'une précision toujours faible. À l'inverse, `attribute.confidentiality` est le cas le plus critique : une seule paire correcte sur 73 attendues (rappel 1,4 %), malgré une précision apparente un peu meilleure (25 %) liée au très petit nombre de propositions.

Ces résultats montrent que la seule similarité sémantique VERIS ↔ ATT&CK ne suffit pas à reproduire le jugement expert. Le faible rappel indique que beaucoup de techniques pertinentes n'apparaissent pas dans le `top-k`, tandis que la faible précision révèle de nombreux faux positifs parmi les candidats proches sémantiquement. Cette baseline motive donc l'introduction d'un contexte complémentaire, via la variante `with_examples`.

4.1.4 Variante `with_examples`

Principe et corpus. La variante `with_examples` reprend l'architecture RAG globale, mais l'enrichit d'un second corpus de contexte : des mappings experts issus de versions antérieures de VERIS / ATT&CK. L'idée est de profiter d'un **transfert analogique** : si une capacité VERIS proche a déjà été mappée par des experts dans le passé, les techniques associées peuvent guider le mapping de la version cible.

Le corpus de retrieval combine donc deux collections ChromaDB :

- **attack_techniques** : le catalogue ATT&CK cible (19.1), comme dans la variante **attack_only**;
- **expert_examples** : les mappings experts des versions antérieures (ATT&CK 9.0, 12.1 et 16.1), agrégés par capacité VERIS.

Le mapping expert de la version cible (VERIS 1.4.1 / ATT&CK 19.1) n'est **jamais** indexé, afin de respecter le principe anti-fuite : les exemples servent uniquement d'analogies, jamais de fuite directe de la référence d'évaluation.

Fonctionnement. Le pipeline reste identique à celui de **attack_only**, avec une étape supplémentaire de retrieval :

1. la capacité VERIS est transformée en requête sémantique ;
2. les **top- k** techniques ATT&CK les plus proches sont récupérées dans **attack_techniques** ;
3. les **top- m** exemples experts les plus proches sont récupérés dans **expert_examples** (paramètre **RAG_TOP_M_EXAMPLES**, 5 par défaut) ;
4. le backend **retrieval** sélectionne les mappings **sans LLM**, en combinant :
 - les candidats ATT&CK dont la similarité dépasse les seuils prédéfinis ;
 - les techniques soutenues par un exemple expert proche ;
 - éventuellement quelques techniques suggérées uniquement par l'exemple le plus proche.
5. validation anti-hallucination et enrichissement depuis le catalogue local ;
6. écriture des sept fichiers JSON dans **Resultat/Resultat_RAG/..._RAG_with_examples/**.

Cette variante permet donc de mesurer l'apport concret des exemples experts par rapport à la baseline **attack_only**, à architecture et backend identiques.

Résultats et remarques. Voici les résultats obtenus par la variante `with_examples` sur la version cible en sortie de notre pipeline expérimental.

```

Solution : Resultat_RAG
Dossier : veris-1.4.1_attack-19.1-enterprise_RAG_with_examples
Version : veris-1.4.1_attack-19.1-enterprise
Experts : veris-1.4.1_attack-19.1-enterprise_json.json
Fichiers : 7/7 scopes presents

GLOBAL (tous scopes agreges) :
  Paires experts=1250 solution=3288 communes= 621
  Precision= 18.9% Rappel= 49.7% F1= 27.4% Jaccard= 15.9%

PAR CAPABILITY_GROUP :

```

scope	exp	sol	comm	Prec	Rapp	F1
action.hacking	534	1256	223	17.8%	41.8%	24.9%
action.malware	405	1298	204	15.7%	50.4%	24.0%
action.social	79	276	50	18.1%	63.3%	28.2%
attribute.integrity	91	208	61	29.3%	67.0%	40.8%
attribute.confidentiality	73	21	18	85.7%	24.7%	38.3%
attribute.availability	44	65	41	63.1%	93.2%	75.2%
value_chain.development	25	164	24	14.6%	96.0%	25.4%

FIGURE 3 – Résultats de la variante `with_examples` sur la version cible VERIS 1.4.1 / ATT&CK 19.1.

La Figure 3 montre un gain marqué par rapport à `attack_only`. Au niveau global, sur les mêmes 1 250 paires expertes, la solution propose 3 288 mappings, dont 621 sont corrects. La précision passe à 18,9 %, le rappel à 49,7 % et le F1 à **27,4 %** (Jaccard 15,9 %). Autrement dit, l'injection d'exemples experts **plus que double le F1**, principalement grâce à une nette amélioration du rappel : près d'une paire experte sur deux est désormais retrouvée.

Le détail par capability group confirme cet apport. Les scopes volumineux `action.hacking` et `action.malware` montent respectivement à 24,9 % et 24,0 % de F1. Les meilleurs résultats s'observent sur `attribute.availability` (F1 75,2 %, rappel 93,2 %) et `attribute.integrity` (F1 40,8 %). `attribute.confidentiality` devient beaucoup plus précis (85,7 %), même si son rappel reste modéré (24,7 %). À l'inverse, `value_chain.development` atteint un rappel très élevé (96 %) mais conserve une précision faible (14,6 %), signe d'une sur-génération encore importante.

Ces résultats montrent que les exemples experts des versions antérieures apportent un contexte utile au mapping, surtout pour retrouver des paires manquées par la seule similarité ATT&CK. Le backend `retrieval` reste toutefois orienté rappel : la précision globale (18,9 %) demeure limitée, car de nombreuses propositions restent incorrectes. Un filtrage plus sélectif, par exemple via un backend LLM, constituerait une piste naturelle d'amélioration.

Et il reste à nuancer que ces résultats sont obtenus sur la version cible VERIS 1.4.1 / ATT&CK 19.1, et que l'on a injecté énormément d'exemples experts issus des versions antérieures : pour une grande partie des capacités, le RAG dispose donc déjà d'analogies proches. Il faudrait également tester l'écart entre le mapping de la version `veris-1.4.0_attack-16.1-enterprise` et celui de la version `veris-1.4.1_attack-19.1-enterprise` afin d'évaluer comment le RAG se comporte sur des cas vraiment nouveaux, absents de sa base d'exemples. Les performances pourraient alors varier selon la disponibilité et la pertinence des exemples experts.

4.1.5 Variante DeepSeek-V4-Flash-0731

Les variantes LLM qui suivent reprennent le pipeline `with_examples`, mais remplacent le backend `retrieval` (seuils de similarité) par un **LLM de sélection** interrogé via l'API Together.ai. Le `retrieval` reste inchangé : pour chaque capacité VERIS, le système fournit au modèle les `top-k` techniques ATT&CK et les `top-m` exemples experts, puis le LLM choisit parmi ces candidats uniquement (garde-fou anti-hallucination). Les trois modèles sont évalués dans les mêmes conditions que précédemment.

Principe et corpus. Cette variante utilise le modèle `deepseek-ai/DeepSeek-V4-Flash-0731` comme backend de décision. Le principe est d'exploiter un LLM relativement rapide (« Flash ») pour filtrer et sélectionner les mappings à partir du contexte récupéré, plutôt que d'appliquer uniquement des seuils de similarité cosinus.

Le corpus reste celui de `with_examples` :

- `attack_techniques` : catalogue ATT&CK cible (19.1) ;
- `expert_examples` : mappings experts des versions antérieures (9.0, 12.1, 16.1), hors version cible.

Le mapping expert de la version évaluée n'est jamais indexé (anti-fuite).

Fonctionnement. Pour chaque capacité VERIS :

1. construction de la requête sémantique (group, label, description) ;
2. `retrieval` des `top-k` techniques ATT&CK et des `top-m` exemples experts ;
3. appel à DeepSeek-V4-Flash via Together.ai (prompt système + prompt utilisateur contenant candidats et exemples) ;
4. parsing JSON de la réponse, avec extraction de secours si le modèle place le JSON dans un bloc de raisonnement ;
5. filtrage strict : seuls les `attack_id` présents dans les candidats de `retrieval` sont conservés ;
6. enrichissement des libellés/tactiques depuis le catalogue local et écriture au format `veris_to_mitre`.

Résultats et remarques. Voici les résultats obtenus par la variante DeepSeek-V4-Flash-0731 en sortie du script d'évaluation.

```
Solution : Resultat_RAG_MultiLLM
Dossier : veris-1.4.1_attack-19.1-enterprise_RAG_MultiLLM_deepseek-v4-flash-0731_with_examples
Version : veris-1.4.1_attack-19.1-enterprise
Experts : veris-1.4.1_attack-19.1-enterprise_json.json
Fichiers : 7/7 scopes presents

GLOBAL (tous scopes agregés) :
Paires experts=1250 solution= 378 communes= 161
Precision= 42.6% Rappel= 12.9% F1= 19.8% Jaccard= 11.0%

PAR CAPABILITY_GROUP :
scope      exp  sol  comm  Prec  Rapp  F1
action.hacking    534  102  43   42.2%  8.1%  13.5%
action.malware    405  139  48   34.5%  11.9%  17.6%
action.social      79   59   23   39.0%  29.1%  33.3%
attribute.integrity 91   38   14   36.8%  15.4%  21.7%
attribute.confidentiality 73   0    0    0.0%  0.0%  0.0%
attribute.availability 44   21   16   76.2%  36.4%  49.2%
value_chain.development 25   19   17   89.5%  68.0%  77.3%
```

FIGURE 4 – Résultats de la variante DeepSeek-V4-Flash-0731 (`with_examples`) sur VERIS 1.4.1 / ATT&CK 19.1.

La Figure 4 présente les résultats obtenus par notre modèle. Au niveau global, sur les 1 250 paires expertes, la solution en propose 378, dont 161 sont correctes. Cela se traduit par une précision de 42,6 %, un rappel de 12,9 % et un F1 de **19,8 %** (Jaccard 11,0 %). Par rapport à `with_examples` en mode `retrieval` (F1 27,4 %, précision 18,9 %), DeepSeek **augmente nettement la précision** (42,6 % contre 18,9 %) mais **réduit le rappel** (12,9 % contre 49,7 %). Le LLM agit donc comme un filtre sélectif : il émet bien moins de propositions (378 contre 3 288) et se trompe moins souvent, au détriment de la couverture. Le détail par capability group montre une forte hétérogénéité. `value_chain.development` atteint un F1 de 77,3 % (précision 89,5 %, rappel 68,0 %), et `attribute.availability` un F1 de 49,2 %. À l'inverse, `attribute.confidentiality` reste à 0 % (aucune paire émise), et les scopes volumineux `action.hacking` et `action.malware` restent modestes (F1 de 13,5 % et 17,6 %). Globalement, DeepSeek illustre bien le compromis précision / rappel lorsque le backend de décision est un LLM plutôt qu'un seuil de similarité.

4.1.6 Variante Llama-3.3-70B-Instruct-Turbo

Principe et corpus. Cette variante utilise le modèle `meta-llama/Llama-3.3-70B-Instruct-Turbo` comme backend de décision. Il s'agit du plus grand des trois modèles retenus (70 milliards de paramètres, variante Turbo serverless sur Together.ai), censé offrir le meilleur suivi d'instructions pour le filtrage des candidats ATT&CK.

Le corpus de retrieval est identique aux autres variantes multi-LLM.

Fonctionnement. Le pipeline est le même que pour DeepSeek : `retrieval with_examples`, puis appel chat au modèle Llama via l'API Together, parsing JSON, filtre strict sur les `attack_id` candidats, et écriture des sept fichiers de scope. Seul change l'identifiant du modèle passé à l'API ; prompts, k , m et post-traitement restent figés pour permettre une comparaison équitable.

Résultats et remarques. Voici les résultats obtenus par la variante Llama-3.3-70B-Instruct-Turbo en sortie du script d'évaluation.

```
Solution : Resultat_RAG_MultiLLM
Dossier : veris-1.4.1_attack-19.1-enterprise_RAG_MultiLLM_llama-3-3-70b-instruct-turbo_with_examples
Version : veris-1.4.1_attack-19.1-enterprise
Experts : veris-1.4.1_attack-19.1-enterprise_json.json
Fichiers : 7/7 scopes presents

GLOBAL (tous scopes agregés) :
  Paires experts=1250 solution= 404 communes= 192
  Precision= 47.5% Rappel= 15.4% F1= 23.2% Jaccard= 13.1%

PAR CAPABILITY_GROUP :
  scope      exp  sol  comm  Prec  Rapp  F1
  action.hacking      534  124   62  50.0%  11.6%  18.8%
  action.malware     405  139   49  35.3%  12.1%  18.0%
  action.social       79   49   23  46.9%  29.1%  35.9%
  attribute.integrity  91   42   19  45.2%  20.9%  28.6%
  attribute.confidentiality 73    4    4 100.0%   5.5%  10.4%
  attribute.availability 44   16   15  93.8%  34.1%  50.0%
  value_chain.development 25   30   20  66.7%  80.0%  72.7%
```

FIGURE 5 – Résultats de la variante Llama-3.3-70B-Instruct-Turbo (`with_examples`) sur VERIS 1.4.1 / ATT&CK 19.1.

La Figure 5 montre les meilleurs scores parmi les trois backends LLM. Globalement, la solution propose 404 paires contre 1250 expertes, dont 192 correctes : précision 47,5 %, rappel 15,4 %, F1 de **23,2 %** (Jaccard 13,1 %).

Llama combine donc une précision encore un peu supérieure à DeepSeek (47,5 % contre 42,6 %) et un rappel légèrement meilleur (15,4 % contre 12,9 %). Le F1 global (23,2 %) dépasse DeepSeek (19,8 %) et Qwen (14,5 %), tout en restant sous le F1 du backend `retrieval_with_examples` (27,4 %), dominé par un rappel bien plus élevé.

Par capability group, les points forts sont `value_chain.development` (F1 72,7 %, rappel 80 %) et `attribute.availability` (F1 50,0 %, précision 93,8 %). Contrairement à DeepSeek, Llama produit au moins quelques paires correctes sur `attribute.confidentiality` (4/4 propositions justes, mais rappel seulement 5,5 %). Les scopes `action.hacking` et `action.malware` restent limités en rappel (11,6 % et 12,1 %), ce qui tire le score global vers le bas malgré une précision autour de 35–50 %. Llama apparaît donc comme le meilleur filtre LLM de notre panel, sans pour autant retrouver le volume de paires correctes de la baseline `retrieval` orientée rappel.

4.1.7 Variante Qwen2.5-7B-Instruct-Turbo

Principe et corpus. Cette variante utilise le modèle Qwen/Qwen2.5-7B-Instruct-Turbo comme backend de décision. Il s'agit d'un modèle nettement plus petit (7 milliards de paramètres) que Llama-70B, retenu pour comparer une famille distincte (Alibaba Qwen) en accès serverless, avec un coût et une latence plus bas.

Le corpus reste celui de `with_examples`. Les sorties sont déposées dans `Resultat_RAG_MultiLLM`, sous le préfixe `..._qwen2-5-7b-instruct-turbo_with_examples`.

Fonctionnement. Le fonctionnement est strictement parallèle aux deux variantes précédentes : même `retrieval`, mêmes prompts, même garde-fou sur les identifiants candidats. Seul le modèle Qwen/Qwen2.5-7B-Instruct-Turbo change, ce qui permet d'isoler l'impact de la taille et de la famille de modèle sur la qualité du mapping.

Résultats et remarques. Voici les résultats obtenus par la variante Qwen2.5-7B-Instruct-Turbo en sortie du script d'évaluation.

```
Solution : Resultat_RAG_MultiLLM
Dossier : veris-1.4.1_attack-19.1-enterprise_RAG_MultiLLM_qwen2-5-7b-instruct-turbo_with_examples
Version : veris-1.4.1_attack-19.1-enterprise
Experts : veris-1.4.1_attack-19.1-enterprise_json.json
Fichiers : 7/7 scopes presents

GLOBAL (tous scopes agregés) :
Paires experts=1250 solution= 203 communes= 105
Precision= 51.7% Rappel= 8.4% F1= 14.5% Jaccard= 7.8%

PAR CAPABILITY_GROUP :
scope      exp  sol  comm  Prec  Rapp  F1
action.hacking      534  73   38  52.1%  7.1% 12.5%
action.malware      405  69   27  39.1%  6.7% 11.4%
action.social        79  17    9  52.9% 11.4% 18.7%
attribute.integrity   91  17    9  52.9%  9.9% 16.7%
attribute.confidentiality 73   2    2 100.0%  2.7%  5.3%
attribute.availability 44   9    7  77.8% 15.9% 26.4%
value_chain.development 25  16   13  81.2% 52.0% 63.4%
```

FIGURE 6 – Résultats de la variante Qwen2.5-7B-Instruct-Turbo (`with_examples`) sur VERIS 1.4.1 / ATT&CK 19.1.

La Figure 6 montre le profil le plus *conservateur* des trois LLM. Sur les 1 250 paires expertes, la solution n'en propose que 203, dont 105 correctes : précision **51,7 %** (la meilleure des trois LLM), rappel 8,4 % et F1 de **14,5 %** (Jaccard 7,8 %).

Qwen maximise donc la confiance des paires émises, au prix d'un rappel très bas : moins d'une paire experte sur douze est retrouvée. Côté scopes, `value_chain.development` reste le meilleur cas (F1 63,4 %), et `attribute.confidentiality` affiche encore 100 % de précision mais sur seulement 2 propositions (rappel 2,7 %). Les scopes `action.hacking` et `action.malware` chutent à 12,5 % et 11,4 % de F1, avec un rappel autour de 7 %.

En synthèse des trois backends LLM : Llama-3.3-70B obtient le meilleur F1 (23,2 %), DeepSeek-V4-Flash un profil intermédiaire (19,8 %), et Qwen2.5-7B la meilleure précision (51,7 %) mais le F1 le plus bas (14,5 %). Tous trois privilégient la précision par rapport au backend `retrieval` de `with_examples`, qui reste supérieur en F1 grâce à son fort rappel. Le choix du LLM apparaît donc comme un régulateur du compromis précision/rappel plutôt que comme un simple gain uniforme.

Ces trois backends restent cependant trop conservateurs : Llama ne retrouve que 192 des 1 250 paires expertes. Nous avons donc poursuivi, sur Llama uniquement, une chaîne d'itérations analogiques (`hybrid fill` / `rerank`) numérotées v2 à v14. L'idée n'est plus de demander au LLM de « découvrir » le mapping, mais de **s'appuyer sur les mappings experts des versions antérieures** pour transférer, filtrer et remettre à jour les identifiants ATT&CK vers la paire cible VERIS 1.4.1 / ATT&CK 19.1.

4.1.8 Itérations analogiques Llama (v2–v14)

Principe commun. Toutes ces versions partagent le même `retrieval with_examples` (ChromaDB + MiniLM) et le même modèle Llama-3.3-70B-Instruct-Turbo. Ce qui change, c'est la **règle locale de décision analogique** appliquée après (ou à la place) de l'appel LLM :

1. récupérer des exemples experts des versions précédentes (jamais la cible, anti-fuite) ;
2. en extraire les identifiants ATT&CK déjà mappés ;

3. en faire l'union, un reclassement à taille constante, ou un budget par capacité, selon la version ;
4. valider chaque identifiant dans le catalogue ATT&CK 19.1 (anti-hallucination).

Le Tableau 3 synthétise l'évolution globale ; les figures suivantes détaillent les résultats obtenue pour chaque version.

Version	Sol.	Comm.	Préc.	Rapp.	F1	Jac.
Llama (v1)	404	192	47,5 %	15,4 %	23,2 %	13,1 %
v2	817	281	34,4 %	22,5 %	27,2 %	15,7 %
v3	1 394	411	29,5 %	32,9 %	31,1 %	18,4 %
v4	940	308	32,8 %	24,6 %	28,1 %	16,4 %
v5	1 418	354	25,0 %	28,3 %	26,5 %	15,3 %
v6	1 306	400	30,6 %	32,0 %	31,3 %	18,6 %
v7	1 028	316	30,7 %	25,3 %	27,7 %	16,1 %
v8	1 316	409	31,1 %	32,7 %	31,9 %	19,0 %
v9	1 316	549	41,7 %	43,9 %	42,8 %	27,2 %
v10	1 307	783	59,9 %	62,6 %	61,2 %	44,1 %
v11	1 185	971	81,9 %	77,7 %	79,8 %	66,3 %
v12	1 379	1 068	77,4 %	85,4 %	81,2 %	68,4 %
v13	1 349	1 127	83,5 %	90,2 %	86,7 %	76,6 %
v14	1 379	1 144	83,0 %	91,5 %	87,0 %	77,0 %

TABLEAU 3 – Évolution des itérations analogiques Llama (with_examples) sur VERIS 1.4.1 / ATT&CK 19.1 (1 250 paires expertes).

v2 — Plus de contexte, toujours le LLM. La v2 élargit le retrieval (davantage de candidats ATT&CK et d'exemples) et durcit le prompt de Llama. Le volume remonte (817 paires contre 404), le F1 passe à **27,2 %**, mais la précision chute (34,4 %) : le LLM reste sous-générateur sur les gros scopes `hacking` / `malware`, tout en sur-proposant sur `value_chain.development`.

```

Solution : Resultat_RAG_MultiLLM
Dossier : veris-1.4.1_attack-19.1-enterprise_RAG_MultiLLM_llama-3-3-70b-instruct-turbo_with_examples_v2
Version : veris-1.4.1_attack-19.1-enterprise
Experts : veris-1.4.1_attack-19.1-enterprise_json.json
Fichiers : 7/7 scopes presents

GLOBAL (tous scopes agregés) :
Paires experts=1250 solution= 817 communes= 281
Precision= 34.4% Rappel= 22.5% F1= 27.2% Jaccard= 15.7%

PAR CAPABILITY_GROUP :
scope      exp  sol  comm  Prec  Rapp  F1
action.hacking  534  265  93   35.1% 17.4% 23.3%
action.malware  405  265  81   30.6% 20.0% 24.2%
action.social   79   128  33   25.8% 41.8% 31.9%
attribute.integrity  91   74   23   31.1% 25.3% 27.9%
attribute.confidentiality  73   9    9   100.0% 12.3% 22.0%
attribute.availability  44   27   19   70.4% 43.2% 53.5%
value_chain.development  25   49   23   46.9% 92.0% 62.2%

```

FIGURE 7 – Résultats de la variante analogique v2 (Llama with_examples).

v3 — Premier fill analogique. Après la décision LLM, la v3 **ajoute** les identifiants ATT&CK présents dans les exemples experts mais omis par le modèle. Le rappel grimpe (32,9 %) et le F1 atteint **31,1 %**, au prix d'une forte sur-génération (1 394 paires, précision 29,5 %), surtout sur `action.social` et `value_chain.development`.

```

Solution : Resultat_RAG_MultiLLM
Dossier : veris-1.4.1_attack-19.1-enterprise_RAG_MultiLLM_llama-3-3-70b-instruct-turbo_with_examples_v3
Version : veris-1.4.1_attack-19.1-enterprise
Experts : veris-1.4.1_attack-19.1-enterprise_json.json
Fichiers : 7/7 scopes presents

GLOBAL (tous scopes agreges) :
Paires experts=1250 solution=1394 communes= 411
Precision= 29.5% Rappel= 32.9% F1= 31.1% Jaccard= 18.4%

PAR CAPABILITY_GROUP :
scope      exp  sol  comm  Prec  Rapp  F1
action.hacking      534  457  138  30.2% 25.8% 27.9%
action.malware      405  483  125  25.9% 30.9% 28.2%
action.social        79   188   42  22.3% 53.2% 31.5%
attribute.integrity  91   118   37  31.4% 40.7% 35.4%
attribute.confidentiality 73   14   13  92.9% 17.8% 29.9%
attribute.availability 44   53   31  58.5% 70.5% 63.9%
value_chain.development 25   81   25  30.9% 100.0% 47.2%

```

FIGURE 8 – Résultats de la variante analogique v3 (fill des IDs experts omis).

v4 à v8 — Plafonds, skips et passes supplémentaires. Les versions suivantes cherchent à calmer cette sur-génération sans perdre le gain de rappel :

- **v4** : plafond d'ajouts par capability group. Le volume redescend (940 paires), F1 **28,1 %**.
- **v5** : pas de fill sur les scopes déjà sur-générés (*social*, *value_chain*), plus d'ajouts ailleurs. Le F1 recule à **26,5 %**.
- **v6** : fill analogique plus strict (sans expansion de famille) : F1 **31,3 %**.
- **v7** : uniquement des IDs issus d'exemples (plus de fill par seule similarité) : F1 **27,7 %**.
- **v8** : seconde passe analogique sur les scopes sous-générés. Meilleur F1 de ce palier : **31,9 %** (1316 paires, 409 communes).

Le constat de ce palier est net : **ajouter** des mappings analogiques ne suffit pas. Tant que l'on empile les IDs du LLM et ceux des exemples, précision et rappel s'opposent. Il faut **reclasser** plutôt qu'accumuler.

```

Solution : Resultat_RAG_MultiLLM
Dossier : veris-1.4.1_attack-19.1-enterprise_RAG_MultiLLM_llama-3-3-70b-instruct-turbo_with_examples_v4
Version : veris-1.4.1_attack-19.1-enterprise
Experts : veris-1.4.1_attack-19.1-enterprise_json.json
Fichiers : 7/7 scopes presents

GLOBAL (tous scopes agreges) :
Paires experts=1250 solution= 940 communes= 308
Precision= 32.8% Rappel= 24.6% F1= 28.1% Jaccard= 16.4%

PAR CAPABILITY_GROUP :
scope      exp  sol  comm  Prec  Rapp  F1
action.hacking      534  319  105  32.9% 19.7% 24.6%
action.malware      405  324   95  29.3% 23.5% 26.1%
action.social        79   138   34  24.6% 43.0% 31.3%
attribute.integrity  91   74   23  31.1% 25.3% 27.9%
attribute.confidentiality 73   9    9  100.0% 12.3% 22.0%
attribute.availability 44   27   19  70.4% 43.2% 53.5%
value_chain.development 25   49   23  46.9% 92.0% 62.2%

```

FIGURE 9 – Résultats de la variante analogique v4 (plafonds d'ajouts par scope).

```

Solution : Resultat_RAG_MultiLLM
Dossier : veris-1.4.1_attack-19.1-enterprise_RAG_MultiLLM_llama-3-3-70b-instruct-turbo_with_examples_v5
Version : veris-1.4.1_attack-19.1-enterprise
Experts : veris-1.4.1_attack-19.1-enterprise_json.json
Fichiers : 7/7 scopes presents

GLOBAL (tous scopes agreges) :
  Paires experts=1250  solution=1418  communes= 354
  Precision= 25.0%  Rappel= 28.3%  F1= 26.5%  Jaccard= 15.3%

PAR CAPABILITY_GROUP :
  scope      exp  sol  comm  Prec  Rapp  F1
  action.hacking      534  527  121  23.0%  22.7%  22.8%
  action.malware      405  539  113  21.0%  27.9%  23.9%
  action.social        79  128  33   25.8%  41.8%  31.9%
  attribute.integrity  91  119  32   26.9%  35.2%  30.5%
  attribute.confidentiality  73  12  11   91.7%  15.1%  25.9%
  attribute.availability  44  44  21   47.7%  47.7%  47.7%
  value_chain.development  25  49  23   46.9%  92.0%  62.2%

```

FIGURE 10 – Résultats de la variante analogique v5 (skip social / value_chain).

```

Solution : Resultat_RAG_MultiLLM
Dossier : veris-1.4.1_attack-19.1-enterprise_RAG_MultiLLM_llama-3-3-70b-instruct-turbo_with_examples_v6
Version : veris-1.4.1_attack-19.1-enterprise
Experts : veris-1.4.1_attack-19.1-enterprise_json.json
Fichiers : 7/7 scopes presents

GLOBAL (tous scopes agreges) :
  Paires experts=1250  solution=1306  communes= 400
  Precision= 30.6%  Rappel= 32.0%  F1= 31.3%  Jaccard= 18.6%

PAR CAPABILITY_GROUP :
  scope      exp  sol  comm  Prec  Rapp  F1
  action.hacking      534  456  139  30.5%  26.0%  28.1%
  action.malware      405  488  124  25.4%  30.6%  27.8%
  action.social        79  128  33   25.8%  41.8%  31.9%
  attribute.integrity  91  118  37   31.4%  40.7%  35.4%
  attribute.confidentiality  73  17  16   94.1%  21.9%  35.6%
  attribute.availability  44  50  28   56.0%  63.6%  59.6%
  value_chain.development  25  49  23   46.9%  92.0%  62.2%

```

FIGURE 11 – Résultats de la variante analogique v6 (fill analogique sans expansion de famille).

```

Solution : Resultat_RAG_MultiLLM
Dossier : veris-1.4.1_attack-19.1-enterprise_RAG_MultiLLM_llama-3-3-70b-instruct-turbo_with_examples_v7
Version : veris-1.4.1_attack-19.1-enterprise
Experts : veris-1.4.1_attack-19.1-enterprise_json.json
Fichiers : 7/7 scopes presents

GLOBAL (tous scopes agreges) :
  Paires experts=1250  solution=1028  communes= 316
  Precision= 30.7%  Rappel= 25.3%  F1= 27.7%  Jaccard= 16.1%

PAR CAPABILITY_GROUP :
  scope      exp  sol  comm  Prec  Rapp  F1
  action.hacking      534  362  109  30.1%  20.4%  24.3%
  action.malware      405  367  99   27.0%  24.4%  25.6%
  action.social        79  128  33   25.8%  41.8%  31.9%
  attribute.integrity  91  84  24   28.6%  26.4%  27.4%
  attribute.confidentiality  73  9   9  100.0%  12.3%  22.0%
  attribute.availability  44  29  19   65.5%  43.2%  52.1%
  value_chain.development  25  49  23   46.9%  92.0%  62.2%

```

FIGURE 12 – Résultats de la variante analogique v7 (IDs analogiques uniquement).


```

Solution : Resultat_RAG_MultiLLM
Dossier : veris-1.4.1_attack-19.1-enterprise_RAG_MultiLLM_llama-3-3-70b-instruct-turbo_with_examples_v8
Version : veris-1.4.1_attack-19.1-enterprise
Experts : veris-1.4.1_attack-19.1-enterprise_json.json
Fichiers : 7/7 scopes presents

GLOBAL (tous scopes agregés) :
Paires experts=1250 solution=1316 communes= 409
Precision= 31.1% Rappel= 32.7% F1= 31.9% Jaccard= 19.0%

PAR CAPABILITY_GROUP :
scope      exp  sol  comm  Prec  Rapp  F1
action.hacking  534  469  145  30.9% 27.2% 28.9%
action.malware  405  485  126  26.0% 31.1% 28.3%
action.social   79   128  33   25.8% 41.8% 31.9%
attribute.integrity  91  118  37   31.4% 40.7% 35.4%
attribute.confidentiality 73   17  17  100.0% 23.3% 37.8%
attribute.availability 44   50  28   56.0% 63.6% 59.6%
value_chain.development 25   49  23   46.9% 92.0% 62.2%

```

FIGURE 13 – Résultats de la variante analogique v8 (seconde passe sur scopes sous-générés).

v9 — Reclassement à N constant. La v9 conserve le **même nombre** de mappings que la v8 (1 316), mais **remplace** les identifiants les plus faibles par de meilleurs candidats analogiques (score de similarité + présence dans les exemples). C'est le premier saut qualitatif : précision 41,7 %, rappel 43,9 %, F1 **42,8 %** (549 communes). Le volume n'augmente pas, seule la *composition* du mapping s'améliore.

```

Solution : Resultat_RAG_MultiLLM
Dossier : veris-1.4.1_attack-19.1-enterprise_RAG_MultiLLM_llama-3-3-70b-instruct-turbo_with_examples_v9
Version : veris-1.4.1_attack-19.1-enterprise
Experts : veris-1.4.1_attack-19.1-enterprise_json.json
Fichiers : 7/7 scopes presents

GLOBAL (tous scopes agregés) :
Paires experts=1250 solution=1316 communes= 549
Precision= 41.7% Rappel= 43.9% F1= 42.8% Jaccard= 27.2%

PAR CAPABILITY_GROUP :
scope      exp  sol  comm  Prec  Rapp  F1
action.hacking  534  469  205  43.7% 38.4% 40.9%
action.malware  405  485  174  35.9% 43.0% 39.1%
action.social   79   128  37   28.9% 46.8% 35.7%
attribute.integrity  91  118  58   49.2% 63.7% 55.5%
attribute.confidentiality 73   17  15   88.2% 20.5% 33.3%
attribute.availability 44   50  36   72.0% 81.8% 76.6%
value_chain.development 25   49  24   49.0% 96.0% 64.9%

```

FIGURE 14 – Résultats de la variante analogique v9 (rerank à N constant).

v10 — Budget analogique et score pondéré. La v10 alloue le nombre de mappings *par capacité* selon la taille de l'exemple analogique le plus proche du même groupe, tout en gardant un N global comparable. Un score pondéré (support des exemples $\times$ similarité) et une déduplication parent/enfant évitent de garder à la fois une technique et toutes ses sous-techniques. Le F1 saute à **61,2 %** (783 communes, précision 59,9 %, rappel 62,6 %). `attribute.confidentiality`, jusqu'ici presque vide, atteint un rappel de 100 % (F1 93,6 %).

```

Solution : Resultat_RAG_MultiLLM
Dossier : veris-1.4.1_attack-19.1-enterprise_RAG_MultiLLM_llama-3-3-70b-instruct-turbo_with_examples_v10
Version : veris-1.4.1_attack-19.1-enterprise
Experts : veris-1.4.1_attack-19.1-enterprise_json.json
Fichiers : 7/7 scopes presents

GLOBAL (tous scopes agregés) :
Paires experts=1250 solution=1307 communes= 783
Precision= 59.9% Rappel= 62.6% F1= 61.2% Jaccard= 44.1%

PAR CAPABILITY_GROUP :
scope      exp  sol  comm  Prec  Rapp  F1
action.hacking      534  481  271  56.3%  50.7%  53.4%
action.malware      405  422  260  61.6%  64.2%  62.9%
action.social        79  104  43   41.3%  54.4%  47.0%
attribute.integrity  91  118  72   61.0%  79.1%  68.9%
attribute.confidentiality  73  83   73   88.0%  100.0%  93.6%
attribute.availability  44  59   41   69.5%  93.2%  79.6%
value_chain.development  25  40   23   57.5%  92.0%  70.8%

```

FIGURE 15 – Résultats de la variante analogique v10 (budget analogique et score pondéré).

v11 — Union same-label. Jusqu'à la v10, l'analogique était « le plus proche du même groupe ». La v11 n'accepte plus que les exemples au **même libellé VERIS** (et le même kind *variety* / *vector*), union de K exemples, en ignorant les labels génériques **Unknown** / **Other**. Le F1 atteint **79,8 %** (971 communes, précision 81,9 %, rappel 77,7 %). Une run complète avec appels LLM (v11_llm) reproduit **exactement les mêmes métriques** que le rerank analogique seul, ce qui confirme que, dès lors que l'analogique same-label existe, le LLM n'apporte plus rien.

```

Solution : Resultat_RAG_MultiLLM
Dossier : veris-1.4.1_attack-19.1-enterprise_RAG_MultiLLM_llama-3-3-70b-instruct-turbo_with_examples_v11
Version : veris-1.4.1_attack-19.1-enterprise
Experts : veris-1.4.1_attack-19.1-enterprise_json.json
Fichiers : 7/7 scopes presents

GLOBAL (tous scopes agregés) :
Paires experts=1250 solution=1185 communes= 971
Precision= 81.9% Rappel= 77.7% F1= 79.8% Jaccard= 66.3%

PAR CAPABILITY_GROUP :
scope      exp  sol  comm  Prec  Rapp  F1
action.hacking      534  461  402  87.2%  75.3%  80.8%
action.malware      405  462  331  71.6%  81.7%  76.4%
action.social        79  65   49  75.4%  62.0%  68.1%
attribute.integrity  91  75   72  96.0%  79.1%  86.7%
attribute.confidentiality  73  68   61  89.7%  83.6%  86.5%
attribute.availability  44  32   32  100.0%  72.7%  84.2%
value_chain.development  25  22   22  100.0%  88.0%  93.6%

```

FIGURE 16 – Résultats de la variante analogique v11 (union same-label).

```

Solution : Resultat_RAG_MultiLLM
Dossier : veris-1.4.1_attack-19.1-enterprise_RAG_MultiLLM_llama-3-3-70b-instruct-turbo_with_examples_v11_llm
Version : veris-1.4.1_attack-19.1-enterprise
Experts : veris-1.4.1_attack-19.1-enterprise_json.json
Fichiers : 7/7 scopes presents

GLOBAL (tous scopes agregés) :
Paires experts=1250 solution=1185 communes= 971
Precision= 81.9% Rappel= 77.7% F1= 79.8% Jaccard= 66.3%

PAR CAPABILITY_GROUP :
scope      exp  sol  comm  Prec  Rapp  F1
action.hacking      534  461  402  87.2%  75.3%  80.8%
action.malware      405  462  331  71.6%  81.7%  76.4%
action.social        79  65   49  75.4%  62.0%  68.1%
attribute.integrity  91  75   72  96.0%  79.1%  86.7%
attribute.confidentiality  73  68   61  89.7%  83.6%  86.5%
attribute.availability  44  32   32  100.0%  72.7%  84.2%
value_chain.development  25  22   22  100.0%  88.0%  93.6%

```

FIGURE 17 – Résultats de la run v11 avec appels LLM (v11_llm) : mêmes scores que le rerank analogique.

v12 — Union complète et LLM en dernier recours. La v12 prend l'union de *tous* les exemples same-label récupérés ($\text{top-}m=48$), conserve les parents analogiques même en présence d'enfants, et n'appelle le LLM que si l'analogue est vide (max 5 IDs). Le rappel monte à 85,4 % et le F1 à **81,2 %**, mais la précision recule (77,4 %) : le fallback LLM, sur les capacités sans analogue, n'a qu'environ 14 % de précision et introduit de nombreux faux positifs.

```
Solution : Resultat_RAG_MultiLLM
Dossier : veris-1.4.1_attack-19.1-enterprise_RAG_MultiLLM_llama-3-3-70b-instruct-turbo_with_examples_v12
Version : veris-1.4.1_attack-19.1-enterprise
Experts : veris-1.4.1_attack-19.1-enterprise_json.json
Fichiers : 7/7 scopes presents

GLOBAL (tous scopes agregés) :
  Paires experts=1250 solution=1379 communes=1068
  Precision= 77.4% Rappel= 85.4% F1= 81.2% Jaccard= 68.4%

PAR CAPABILITY_GROUP :
```

scope	exp	sol	comm	Prec	Rapp	F1
action.hacking	534	504	418	82.9%	78.3%	80.5%
action.malware	405	504	363	72.0%	89.6%	79.9%
action.social	79	121	65	53.7%	82.3%	65.0%
attribute.integrity	91	102	88	86.3%	96.7%	91.2%
attribute.confidentiality	73	80	73	91.2%	100.0%	95.4%
attribute.availability	44	42	42	100.0%	95.5%	97.7%
value_chain.development	25	26	17	65.4%	68.0%	66.7%

FIGURE 18 – Résultats de la variante analogique v12 (union complète et LLM si analogue vide).

v13 — Analogie pure, y compris Unknown/Other. La v13 abandonne le LLM. Si un analogue same-label existe — y compris pour *Unknown / Other / NA* — on prend toute l'union ; sinon on s'abstient. C'est la meilleure précision de la série (83,5 %) avec un rappel de 90,2 % et un F1 de **86,7 %** (1 127 communes). La limite devient explicite : la solution **transfère** l'historique expert, elle ne **découvre** pas un mapping inédit.

```
Solution : Resultat_RAG_MultiLLM
Dossier : veris-1.4.1_attack-19.1-enterprise_RAG_MultiLLM_llama-3-3-70b-instruct-turbo_with_examples_v13
Version : veris-1.4.1_attack-19.1-enterprise
Experts : veris-1.4.1_attack-19.1-enterprise_json.json
Fichiers : 7/7 scopes presents

GLOBAL (tous scopes agregés) :
  Paires experts=1250 solution=1349 communes=1127
  Precision= 83.5% Rappel= 90.2% F1= 86.7% Jaccard= 76.6%

PAR CAPABILITY_GROUP :
```

scope	exp	sol	comm	Prec	Rapp	F1
action.hacking	534	520	461	88.7%	86.3%	87.5%
action.malware	405	522	383	73.4%	94.6%	82.6%
action.social	79	75	59	78.7%	74.7%	76.6%
attribute.integrity	91	87	84	96.6%	92.3%	94.4%
attribute.confidentiality	73	80	73	91.2%	100.0%	95.4%
attribute.availability	44	42	42	100.0%	95.5%	97.7%
value_chain.development	25	23	23	100.0%	92.0%	95.8%

FIGURE 19 – Résultats de la variante analogique v13 (union same-label sans LLM).

v14 — Remap d'identifiants et découverte conservative. Le vrai problème restant n'est plus « trouver un analogue », mais **porter l'historique vers ATT&CK 19.1**. Plusieurs IDs anciens ont changé de code (par exemple T1070.001 Clear Windows Event Logs devient T1685.005). La v14 fait donc trois choses :

1. lookup same-label dans **tout le corpus** d'exemples (plus seulement le $\text{top-}m$ vectoriel) ;
2. **remap flou** des IDs historiques vers le catalogue 19.1 (correspondance exacte, puis similarité de nom, puis parent) ;

- pour les labels vraiment nouveaux : découverte conservative par retrieval ATT&CK (similarité $\geq 0,55$, au plus un ID, filtre de tactique).

Le F1 atteint **87,0 %** (1144 communes, précision 83,0 %, rappel 91,5 %, Jaccard 77,0 %). Log `tampering`, vide en v13 à cause d'IDs obsolètes, est à nouveau mappé. Les attributs `confidentiality` et `availability` restent à F1 95–98 %. Le résidu d'erreur se concentre surtout sur quelques unions analogiques trop larges (`malware.variety.backdoor_or_c2`, `hacking.variety.use_of_stolen_creds`) et sur les labels sans aucun historique mappé.

```
Solution : Resultat_RAG_MultiLLM
Dossier  : veris-1.4.1_attack-19.1-enterprise_RAG_MultiLLM_llama-3-3-70b-instruct-turbo_with_examples_v14
Version  : veris-1.4.1_attack-19.1-enterprise
Experts  : veris-1.4.1_attack-19.1-enterprise_json.json
Fichiers : 7/7 scopes presents

GLOBAL (tous scopes agregés) :
Paires experts=1250 solution=1379 communes=1144
Precision= 83.0% Rappel= 91.5% F1= 87.0% Jaccard= 77.0%

PAR CAPABILITY_GROUP :
scope      exp  sol  comm  Prec  Rapp  F1
action.hacking      534  531  468  88.1%  87.6%  87.9%
action.malware      405  531  389  73.3%  96.0%  83.1%
action.social        79   79   60  75.9%  75.9%  75.9%
attribute.integrity   91   93   87  93.5%  95.6%  94.6%
attribute.confidentiality  73   80   73  91.2% 100.0%  95.4%
attribute.availability  44   42   42 100.0%  95.5%  97.7%
value_chain.development 25   23   23 100.0%  92.0%  95.8%
```

FIGURE 20 – Résultats de la variante analogique v14 (corpus same-label, remap ATT&CK et découverte conservative).

4.2 Solution Fine-tune

4.3 Solution PROMPT

4.4 Solution Fine-tune

4.4.1 Principe général

La solution **Fine-tune** vise à générer automatiquement des mappings entre les capacités VERIS et les techniques MITRE ATT&CK à partir des mappings experts disponibles. Contrairement à une approche par simple prompt, cette solution exploite les associations déjà établies par les experts afin d'apprendre une relation entre la description textuelle d'une capacité VERIS et les techniques ATT&CK correspondantes.

Dans le cadre de ce projet, le terme *fine-tune* désigne une approche supervisée locale légère. Il ne s'agit pas d'un réentraînement complet d'un grand modèle de langage de type Llama, Mistral ou Gemma avec LoRA ou QLoRA. Le choix retenu repose plutôt sur une chaîne classique de traitement automatique du langage combinant une vectorisation TF-IDF, une classification multi-label et une régression logistique.

Ce choix présente plusieurs avantages dans le contexte du projet. D'une part, le volume de données annotées est limité, ce qui rend difficile l'entraînement efficace d'un grand modèle de langage. D'autre part, une approche locale est plus simple à reproduire, moins coûteuse et indépendante d'une API externe. Elle permet également de produire directement des fichiers JSON structurés, compatibles avec le comparateur du projet.

4.4.2 Données utilisées

La solution utilise les fichiers préparés dans le dossier :

`data_for_work/attack-19.1_veris-1.4.1/`

Trois fichiers sont nécessaires au fonctionnement du pipeline :

- `veris_1.4.1.json` : capacités VERIS à mapper ;
- `attack_19.1.json` : techniques et sous-techniques MITRE ATT&CK candidates ;
- `mapping_des_experts.json` : mappings experts utilisés comme référence supervisée.

Le fichier `mapping_des_experts.json` permet de construire les exemples d'entraînement. Chaque capacité VERIS est associée à une ou plusieurs techniques ATT&CK. La tâche est donc formulée comme un problème de classification multi-label : une même entrée VERIS peut correspondre à plusieurs sorties MITRE.

Les sept capability groups traités sont ceux attendus par le projet :

```
action.hacking
action.malware
action.social
attribute.availability
attribute.confidentiality
attribute.integrity
value_chain.development
```

Chaque groupe donne lieu à un fichier JSON séparé au format `veris_to_mitre`.

4.4.3 Pipeline supervisé local

Le fichier principal de cette solution est :

`Solution/Solution_FINE_TUNE/run_mapping.py`

Architecture logicielle. La solution Fine-tune est volontairement plus légère que les approches RAG et PROMPT, mais elle suit tout de même une architecture structurée. Le script principal regroupe les étapes de chargement des données, de construction du jeu d'entraînement, d'apprentissage du modèle, de prédiction et d'écriture des fichiers de sortie.

Composant	Rôle
<code>run_mapping.py</code>	Orchestration complète du pipeline Fine-tune local
<code>mapping_des_experts.json</code>	Référence utilisée pour construire les labels supervisés
<code>veris_1.4.1.json</code>	Capacités VERIS transformées en représentations textuelles
<code>attack_19.1.json</code>	Catalogue des techniques MITRE ATT&CK candidates
<code>TfidfVectorizer</code>	Transformation des textes VERIS en vecteurs numériques
<code>MultiLabelBinarizer</code>	Encodage multi-label des techniques MITRE associées
<code>OneVsRestClassifier</code>	Décomposition du problème multi-label en décisions binaires
<code>LogisticRegression</code>	Modèle de décision utilisé pour prédire les techniques ATT&CK

TABLEAU 4 – Composants principaux de la solution Fine-tune locale.

Le pipeline suit plusieurs étapes. Les données VERIS, ATT&CK et les mappings experts sont d'abord chargés depuis le dossier de travail. Les capacités VERIS sont ensuite transformées en représentations textuelles à partir de leur identifiant, de leur capability group,

de leur label et de leur description. Cette représentation permet de fournir au modèle un contexte textuel suffisant pour apprendre des rapprochements sémantiques.

Les mappings experts sont ensuite convertis en un jeu d'entraînement. La partie textuelle des capacités VERIS constitue l'entrée du modèle, tandis que les identifiants MITRE associés constituent les labels à prédire. Comme une capacité VERIS peut correspondre à plusieurs techniques MITRE, les labels sont encodés avec un `MultiLabelBinarizer`.

Le modèle repose sur la chaîne suivante :

- vectorisation TF-IDF des textes VERIS ;
- classification multi-label avec une stratégie One-vs-Rest ;
- régression logistique pour prédire les techniques ATT&CK ;
- filtrage des prédictions selon un seuil de confiance ;
- limitation du nombre de techniques conservées avec un paramètre `top-k`.

La sortie générée respecte le format commun du projet. Pour chaque capability group, un fichier JSON est produit :

```
action.hacking.json
action.malware.json
action.social.json
attribute.availability.json
attribute.confidentiality.json
attribute.integrity.json
value_chain.development.json
```

Découpage par capability group. Le traitement est réalisé séparément pour chaque capability group VERIS. Ce choix répond d'abord à une contrainte de format imposée par le projet : chaque solution doit produire sept fichiers JSON distincts, un pour chaque groupe de capacités VERIS. Cette organisation permet ensuite au comparateur d'évaluer les résultats de manière structurée et homogène.

Le découpage par capability group présente également un intérêt méthodologique. Les capacités VERIS ne décrivent pas toutes le même type de comportement ou d'impact. Par exemple, `action.hacking` regroupe des actions liées aux techniques de compromission, `action.malware` concerne les comportements liés aux logiciels malveillants, tandis que `attribute.availability` décrit des impacts sur la disponibilité. Traiter ces groupes séparément permet donc de conserver cette organisation sémantique dans les fichiers générés.

Dans le cas du modèle local supervisé, ce découpage permet de générer des résultats plus lisibles et plus faciles à analyser. Chaque fichier de sortie contient uniquement les mappings associés à un groupe donné, ce qui facilite l'identification des catégories où le modèle fonctionne correctement et de celles où les erreurs sont plus fréquentes.

Dans le cas de la variante LLM zero-shot, ce découpage est encore plus important. Envoyer l'ensemble des capacités VERIS dans un seul prompt produirait un contexte trop long et augmenterait le risque de réponses mal structurées. Le traitement par capability group limite donc la taille des entrées envoyées au modèle et améliore la stabilité du format JSON généré.

Ce découpage permet enfin de comparer plus facilement les approches entre elles. Les solutions RAG, PROMPT et FINE_TUNE produisent toutes les mêmes sept fichiers de sortie, ce qui garantit une évaluation équitable avec le script de comparaison.

Les fichiers sont écrits dans :

Resultat/Resultat_FINE_TUNE/

4.4.4 Paramètres de décision

Deux paramètres influencent directement le comportement du modèle local :

- **threshold** : seuil minimal de confiance pour conserver une prédiction ;
- **top-k** : nombre maximal de techniques ATT&CK conservées par capacité VERIS.

Un seuil élevé rend le modèle plus strict. Il génère moins de mappings, ce qui peut augmenter la précision, mais réduit généralement le rappel. À l'inverse, un seuil plus faible rend le modèle plus permissif : il retrouve davantage de correspondances expertes, mais peut aussi générer davantage de faux positifs.

Le paramètre **top-k** agit comme une limite sur le nombre de prédictions conservées. Il permet d'éviter que le modèle associe trop de techniques ATT&CK à une même capacité VERIS, tout en laissant suffisamment de possibilités pour retrouver les paires expertes.

La configuration retenue pour le meilleur résultat local est :

```
python Solution/Solution_FINE_TUNE/run_mapping.py --threshold 0.05 --top-k 20 --output
```

4.4.5 Limites de l'approche locale

Même si le modèle supervisé local obtient les meilleurs résultats dans cette partie, il présente plusieurs limites. La première concerne sa dépendance aux mappings experts utilisés pour l'entraînement : le modèle apprend principalement à reproduire les associations déjà présentes dans les données, et peut donc avoir plus de difficulté à généraliser vers des capacités très différentes ou vers de nouvelles versions du framework.

La seconde limite vient de la représentation textuelle utilisée. La vectorisation TF-IDF capture les mots et groupes de mots importants, mais elle ne modélise pas le sens profond d'une phrase comme le ferait un modèle de langage moderne. Deux descriptions sémantiquement proches mais formulées avec un vocabulaire différent peuvent donc être moins bien rapprochées.

Enfin, le choix des paramètres **threshold** et **top-k** influence fortement le compromis entre précision et rappel. Un réglage trop strict réduit les faux positifs mais manque davantage de mappings experts, tandis qu'un réglage trop permissif augmente le rappel au prix d'une précision plus faible. La configuration retenue correspond donc à un compromis empirique, déterminé à partir des résultats obtenus avec le script de comparaison.

4.4.6 Variante LLM zero-shot

En complément du modèle supervisé local, une variante basée sur un LLM déjà entraîné a été expérimentée. Cette variante est implémentée dans :

```
Solution/Solution_FINE_TUNE/run_mapping_llm.py
```

Elle utilise des modèles LLM via l'API Together AI. Le modèle principalement retenu est :

```
meta-llama/Llama-3.3-70B-Instruct-Turbo
```

Ce modèle n'est pas réentraîné dans le cadre du projet. Il est utilisé en *zero-shot*, c'est-à-dire uniquement à partir d'un prompt décrivant la tâche à effectuer. L'objectif est de comparer une approche supervisée locale avec une approche LLM généraliste déjà instruction-tuned.

Le script LLM reprend la même logique de sortie que le modèle local : sept fichiers JSON sont générés, un par capability group VERIS. Pour limiter la taille des prompts, les capacités VERIS sont traitées par batchs. Le script permet également de choisir le modèle, de limiter le nombre d'éléments traités et de contrôler le nombre de candidats ATT&CK fournis au modèle.

Provenance des modèles utilisés. Les modèles utilisés dans cette variante ne sont pas développés directement dans le cadre du projet. Ils proviennent de familles de modèles pré-entraînés et instruction-tuned publiées par des acteurs externes, puis sont appelés via la plateforme Together AI. Dans notre cas, Together AI joue le rôle de fournisseur d'API et d'environnement d'exécution, tandis que les modèles eux-mêmes proviennent de leurs éditeurs respectifs.

Le modèle `meta-llama/Llama-3.3-70B-Instruct-Turbo` appartient à la famille Llama développée par Meta. Il s'agit d'un modèle de langage de 70 milliards de paramètres, pré-entraîné puis adapté au suivi d'instructions. Il est utilisé ici uniquement en inférence, sans réentraînement ni fine-tuning supplémentaire.

Le modèle `Qwen/Qwen2.5-7B-Instruct-Turbo`, testé en complément, appartient à la famille Qwen développée par Alibaba Cloud. Il s'agit également d'un modèle instruction-tuned, utilisé via Together AI afin de comparer son comportement avec celui de Llama sur la même tâche de génération de mappings VERIS vers MITRE ATT&CK.

Les principaux paramètres du script LLM sont :

- `-model` : choix du modèle LLM ;
- `-batch-size` : nombre de capacités VERIS envoyées dans un même appel ;
- `-top-k-attack` : nombre de techniques ATT&CK candidates fournies au modèle ;
- `-limit-per-scope` : limite optionnelle du nombre d'éléments traités par scope ;
- `-output-name` : nom du dossier de sortie généré dans les résultats.

Une configuration de référence a d'abord été obtenue avec :

```
python Solution/Solution_FINE_TUNE/run_mapping_llm.py --model meta-llama/Llama-3.3-70B-Instruct-Turbo
```

Des tests complémentaires ont ensuite été réalisés avec davantage de candidats MITRE.

La meilleure configuration Llama obtenue est :

```
python Solution/Solution_FINE_TUNE/run_mapping_llm.py --model meta-llama/Llama-3.3-70B-Instruct-Turbo
```

4.4.7 Paramétrage expérimental

Le script LLM a été conçu pour faciliter la comparaison de plusieurs configurations expérimentales sans modifier directement le code source. Les principaux paramètres du pipeline peuvent être définis depuis la ligne de commande, ce qui permet de reproduire plus facilement les essais et de comparer différents modèles ou réglages.

Une première configuration de référence a été lancée avec `-batch-size 2` et `-top-k-attack 30`. Elle correspond au dossier `FINE_TUNED_LLM_LLAMA_B2_K30`. Cette configuration a servi de base de comparaison pour les tests suivants.

Les expérimentations complémentaires ont ensuite augmenté le nombre de techniques MITRE candidates fournies au modèle. La configuration B2_K40 utilise toujours des batches de deux capacités VERIS, mais fournit quarante techniques MITRE candidates par appel. Cette configuration a permis d'obtenir le meilleur résultat Llama en F1.

Des variantes plus strictes ont également été testées en limitant, lors d'une étape de post-traitement, le nombre de mappings MITRE conservés pour chaque capacité VERIS. L'objectif était de réduire le nombre total de solutions générées tout en essayant de conserver un maximum de paires communes avec les mappings experts.

Le paramètre `-batch-size` agit principalement sur la stabilité des réponses. Un batch trop grand augmente la taille du prompt et peut provoquer des réponses JSON moins fiables. À l'inverse, un batch plus petit améliore la lisibilité du prompt, mais augmente le nombre d'appels API nécessaires.

Le paramètre `-top-k-attack` influence le nombre de techniques candidates proposées au modèle. Une valeur trop faible risque d'exclure la bonne technique MITRE avant même la génération. Une valeur trop élevée augmente la taille du prompt et peut rendre la décision du modèle moins précise. Le choix de ce paramètre correspond donc à un compromis entre couverture, coût d'appel et qualité de la réponse.

Enfin, le paramètre `-output-name` permet de conserver plusieurs expériences dans des dossiers séparés. Cela évite d'écraser les anciens résultats et facilite ensuite la comparaison automatique des configurations avec le script d'évaluation.

4.4.8 Robustesse du format de sortie

Une difficulté importante des approches fondées sur les LLM concerne la stabilité du format de sortie. Même lorsque le prompt impose une réponse au format JSON strict, le modèle peut produire une réponse partiellement invalide. Il peut par exemple ajouter du texte explicatif avant ou après le JSON, modifier le nom des clés attendues, oublier certains champs, ou retourner une chaîne de caractères à la place d'un objet structuré.

Dans ce projet, cette contrainte est particulièrement importante, car l'évaluation des résultats est entièrement automatisée. Le comparateur ne vérifie pas seulement la pertinence sémantique des mappings générés : il attend également une structure précise au format `veris_to_mitre`. Une réponse compréhensible par un humain peut donc être inutilisable par le pipeline si elle ne respecte pas strictement le schéma attendu.

Par exemple, certaines réponses LLM peuvent retourner une valeur isolée comme :

```
"action.malware.vector.Remote injection"
```

alors que le format attendu doit contenir une entrée complète avec les informations VERIS, la liste des mappings MITRE associés et un indicateur précisant si aucun mapping n'a été trouvé.

Pour limiter ce problème, des scripts de correction et de normalisation ont été utilisés :

```
normalize_llm_outputs.py  
fix_bad_llm_entries.py  
fix_bad_llama_entries.py
```

Ces scripts ont pour objectif de rendre les fichiers générés compatibles avec le comparateur. Ils peuvent notamment corriger certaines structures invalides, harmoniser les noms de champs, extraire les identifiants MITRE lorsqu'ils sont présents dans une réponse mal structurée, et convertir les sorties vers le format attendu.

Cette étape ne consiste pas à inventer de nouveaux mappings. Elle vise uniquement à rendre exploitables les réponses déjà produites par le modèle. Les corrections portent donc sur la forme des fichiers JSON, et non sur le contenu sémantique des associations VERIS-MITRE.

La robustesse du format de sortie constitue ainsi une étape indispensable du pipeline LLM. Sans cette normalisation, une partie des résultats générés ne pourrait pas être évaluée automatiquement, même lorsque les réponses contiennent des informations potentiellement pertinentes.

4.4.9 Contraintes liées à l'utilisation d'une API externe

L'expérimentation avec un LLM zero-shot repose sur l'utilisation de l'API Together AI. Contrairement au modèle local supervisé, cette approche dépend donc d'un service externe pour générer les mappings. Cette dépendance introduit plusieurs contraintes pratiques : disponibilité de l'API, temps de réponse, coût des appels, quotas de crédits et stabilité des réponses retournées par le modèle.

La clé API est chargée depuis un fichier local, par exemple :

```
dev.env
```

Ce fichier n'est pas destiné à être ajouté au dépôt GitHub, car il contient des informations sensibles. Les fichiers de configuration locale comme `dev.env`, `.env` ou `.dev.env` doivent donc rester exclus du suivi Git.

L'utilisation d'une API externe a également un impact sur la reproductibilité des expériences. Une même commande peut être relancée localement, mais elle nécessite toujours que la clé API soit valide et que le compte dispose de crédits suffisants. Cela distingue cette approche du modèle supervisé local, qui peut être exécuté sans appel réseau une fois les données et les dépendances installées.

Cette contrainte est apparue lors d'une première tentative d'amélioration de la configuration Llama. Une expérience supplémentaire avait été préparée avec la configuration suivante :

```
FINE_TUNED_LLM_LLAMA_B2_K40_MAX2
```

L'objectif était d'augmenter le nombre de candidats MITRE ATT&CK fournis au modèle avec `-top-k-attack 40`, tout en limitant le nombre de mappings conservés par capacité VERIS. Cette stratégie devait permettre de réduire le nombre total de solutions générées tout en essayant d'augmenter les paires communes avec les mappings experts.

Une première exécution n'a pas pu aboutir, car l'API Together a renvoyé l'erreur suivante :

```
402 Credit limit exceeded
```

Après l'obtention de nouveaux crédits API, l'expérience a toutefois pu être relancée et évaluée. Les résultats montrent que la limitation du nombre de mappings réduit bien le nombre total de solutions générées, mais qu'elle peut également supprimer des mappings corrects. Cette contrainte illustre donc à la fois la dépendance aux crédits API et l'importance d'évaluer expérimentalement les compromis entre précision, rappel et nombre de solutions générées.

Cette limite montre que les approches LLM externalisées ne doivent pas être évaluées uniquement selon leurs performances théoriques. Leur coût, leur dépendance à un fournisseur externe, les quotas disponibles et la stabilité des réponses doivent également être pris en compte. Dans ce contexte, le modèle local supervisé présente un avantage important : il reste entièrement reproductible localement et ne dépend pas d'un service tiers au moment de l'exécution.

4.5 Solution PROMPT

4.5.1 Principe

La solution **PROMPT** (dossier `Solution/Solution_PROMPT/`) s'appuie sur un LLM généraliste interrogé via l'API **Together AI**, sans index vectoriel ni fine-tuning. L'approche est purement fondée sur le *prompt engineering* en mode **zero-shot** : un *system prompt* fixe le rôle, les règles anti-hallucination et le schéma JSON attendu, un *user prompt* fournit, pour chaque capability group VERIS, les données de référence et l'instruction de mapping.

Contrairement à la solution RAG, aucun contexte n'est récupéré dynamiquement : le modèle reçoit les bases de données préparées (`veris_<v>.json` et `attack_<a>.json`) (les données sont filtrées afin d'alléger le nombre de tokens en entrée) et doit produire des **liens d'identifiants** (`veris_id` → liste d'`attack_ids`).

Les libellés, les tactiques et les sens MITRE → VERIS sont ensuite reconstruits localement à partir des bases de données (`veris_<v>.json` et `attack_<a>.json`) sur le même principe que l'enrichissement post-génération du RAG, avec le script `reconstruct_mapping.py`.

4.5.2 Périmètre traité

Le traitement est découpé selon les sept capability groups du mapping expert :

- `action.hacking`, `action.malware`, `action.social`
- `attribute.integrity`, `attribute.confidentiality`, `attribute.availability`
- `value_chain.development`

Une requête LLM est émise **par capability group**. Les sept réponses sont ensuite écrites dans `Resultat/Resultat_PROMPT/veris-<v>_attack-<a>-enterprise_PROMPT/`, sous la forme de fichiers `<capability>.json`.

4.5.3 Architecture logicielle

Le pipeline est implémenté dans le script unique `run_mapping.py` :

1. chargement des entrées (`-dw` ou couple `-a/-v`) (par défaut, on prend les dernières version des bases de données)
2. filtrage VERIS au groupe courant et compactage ATT&CK (description tronquées à 250 caractères)
3. construction d'un user prompt par capability et chargement du `system-prompt.md`
4. génération LLM séquentielle (timeout 900 s, `max_tokens=100000`)
5. écriture des JSON bruts (IDs) dans `_raw/`
6. reconstruction locale du mapping complet (enrichissement + inversion `mitre_to_veris`) directement dans les `*.json` du dossier de résultats, au format attendu par les scripts de comparaison.

Élément	Rôle
<code>run_mapping.py</code>	Orchestration, prompts, appels API, écriture
<code>reconstruct_mapping.py</code>	Enrichissement local IDs, production JSON finaux
<code>system-prompt.md</code>	Rôle CTI, règles, schéma JSON de sortie
<code>requirements.txt</code>	Dépendance <code>together</code>

TABLEAU 5 – Composants de la solution PROMPT

4.5.4 System Prompt

Le fichier `system-prompt.md` fixe le rôle CTI du LLM (VERIS vs ATT&CK), le mode **zero-shot** et les règles anti-hallucination : n'utiliser que les IDs fournis dans le user prompt, autoriser l'absence de mapping, et interdire toute invention d'identifiants. La sortie LLM est volontairement minimale : uniquement des paires `veris_id` → `attack_ids`, avec `mitre_to_veris` laissé vide (reconstruction locale ensuite). Aucun plafond artificiel n'est imposé sur le nombre de techniques associées à une variety VERIS.

4.5.5 Modèles Together AI

Deux modèles serverless ont été retenus :

`moonshotai/Kimi-K3` et `deepseek-ai/DeepSeek-V4-Pro`. Ils sont aujourd'hui les modèles les plus performant sur TogetherIA. De plus, ils ont été choisis pour leur grande fenêtre de contexte adaptée à nos prompts volumineux

4.5.6 Problèmes rencontrés

Initialement, les catalogues étaient envoyés entiers vers les modèles, la consigne stipulant de faire un mapping bidirectionnel en réponse. Ce qui a fait que les premières runs se sont conclues par des timeouts, des fichiers vides et des sorties tronquées.

4.5.7 Correctifs effectués

Les principaux correctifs effectués afin de rectifier les problèmes ont été de :

- réduction des données d'entrée (filtre VERIS + ATT&CK compact)
- réduction de la sortie LLM aux seuls IDs
- reconstruction post-génération du mapping complet pour la comparaison avec le mapping expert
- traitement séquentiel, validation JSON, conservation des bruts dans `_raw/`

4.5.8 Limites méthodologiques

Plusieurs choix techniques bornent l'interprétation des scores.

La génération est "zero-shot" : contrairement à la variante RAG aucun mapping expert antérieur n'est fourni comme exemple. Les IDs ATT&CK inconnus du catalogue local sont rejetés à la reconstruction. Les descriptions ATT&CK sont tronquées à 250 caractères, ce qui peut appauvrir le signal sémantique sur les techniques longues. Enfin, la dépendance à une API serverless partagée (timeouts, rate limits) a fortement conditionné la faisabilité du premier pipeline et justifie le traitement séquentiel.

4.6 Synthèse des trois solutions

5 Résultats

5.1 Configuration expérimentale

5.2 Résultats — Solution RAG

Pour rappel la règle locale de décision analogique est l'étape qui, pour chaque capacité VERIS, choisit les identifiants ATT&CK à partir des mappings experts déjà connus, plutôt qu'en demandant au LLM d'inventer le mapping. Après le retrieval (ChromaDB + MiniLM), le système s'appuie uniquement sur des exemples issus des versions antérieures — jamais sur la paire cible VERIS 1.4.1 / ATT&CK 19.1, afin d'éviter toute fuite. Il en extrait les IDs déjà associés au même label VERIS (ou à un voisin), puis applique une règle déterministe : union des IDs, reclassement à taille constante, ou budget analogique par capacité, selon la version. Un identifiant n'est retenu que s'il existe encore dans le catalogue ATT&CK 19.1, soit à l'identique, soit après le remap flou de nom ou de parent introduit en v14. La règle est dite *locale* parce qu'elle opère capacité par capacité, sans apprentissage global : elle transfère, filtre et remet à jour l'historique expert, au lieu de découvrir un mapping de zéro.

C'est ce levier — et non le LLM — qui produit les meilleurs résultats de l'étude. Llama seul plafonne à un F1 d'environ 23 % ; la configuration analogique v14 atteint une précision de 83,0 %, un rappel de 91,5 % et un F1 de **87,0 %** sur les 1 250 paires expertes. Les attributs (*confidentiality*, *availability*, *integrity*) et *value_chain.development* dépassent même 94 % de F1. Sur le protocole actuel, la solution RAG est donc nettement supérieure aux baselines retrieval et aux backends LLM seuls.

Ces scores globaux restent toutefois difficiles à interpréter tant qu'on ne distingue pas ce qui a *changé* d'une version à l'autre. Une large partie du mapping VERIS 1.4.1 / ATT&CK 19.1 peut être identique à celui de VERIS 1.4.0 / ATT&CK 16.0. Dans ce cas, un F1 élevé peut simplement refléter la recopie de l'historique stable, ce qui a peu d'intérêt opérationnel. Il faudrait donc réévaluer la solution **uniquement sur les mappings qui diffèrent** entre ces deux versions, afin de mesurer sa capacité réelle à suivre les écarts de vocabulaire, les IDs ATT&CK déplacés et les labels VERIS nouveaux ou révisés. C'est sur ce sous-ensemble — et non sur l'ensemble des 1 250 paires — que l'on pourra trancher si le RAG analogique apporte une valeur de migration, ou s'il se contente de transférer ce qui n'a pas bougé.

5.3 Résultats — Solution Fine-tune

5.3.1 Résultats du modèle supervisé local

La première série d'expérimentations concerne le modèle supervisé local basé sur TF-IDF, One-vs-Rest et régression logistique. Plusieurs configurations ont été testées en faisant varier le seuil de confiance et le nombre maximal de techniques MITRE conservées par capacité VERIS.

Configuration	Solutions	Communes	Précision	Rappel	F1	Jaccard
FINE_TUNE	209	123	58.9 %	9.8 %	16.9 %	9.2 %
FINE_TUNE_T010_K10	344	234	68.0 %	18.7 %	29.4 %	17.2 %
FINE_TUNE_T0055_K15	1117	469	42.0 %	37.5 %	39.6 %	24.7 %
FINE_TUNE_T005_K10	1315	465	35.4 %	37.2 %	36.3 %	22.1 %
FINE_TUNE_T005_K15	1467	539	36.7 %	43.1 %	39.7 %	24.7 %
FINE_TUNE_T005_K20	1541	576	37.4 %	46.1 %	41.3 %	26.0 %

TABLEAU 6 – Résultats globaux du modèle supervisé local Fine-tune

La meilleure configuration du modèle local est FINE_TUNE_T005_K20. Elle obtient un F1 de 41.3 %, avec une précision de 37.4 % et un rappel de 46.1 %. Cette configuration ne présente pas la meilleure précision brute, mais elle offre le meilleur équilibre global entre précision et rappel. Elle retrouve 576 paires communes avec les mappings experts, ce qui en fait le meilleur résultat obtenu dans la partie Fine-tune.

5.3.2 Résultats de la variante LLM zero-shot

En complément du modèle local, plusieurs essais ont été réalisés avec Llama 3.3 70B Instruct Turbo via l'API Together. Le modèle n'a pas été réentraîné dans ce projet : il est utilisé en zero-shot, uniquement à partir d'un prompt et d'une liste de techniques MITRE candidates.

Une première configuration de référence, B2_K30, avait été obtenue avec un batch size de 2 et 30 techniques MITRE candidates. Des tests complémentaires ont ensuite été réalisés afin d'essayer d'obtenir davantage de paires communes avec les experts tout en réduisant le nombre total de mappings générés. Pour cela, le nombre de candidats MITRE a été augmenté, puis des variantes plus strictes ont été évaluées en limitant, lors d'une étape de post-traitement, le nombre de mappings conservés par capacité VERIS.

Configuration Llama	Solutions	Communes	Précision	Rappel	F1	Jaccard
BATCH5	286	39	13.6 %	3.1 %	5.1 %	2.6 %
B3_K25	507	78	15.4 %	6.2 %	8.9 %	4.6 %
B2_K30	471	80	17.0 %	6.4 %	9.3 %	4.9 %
B2_K40	562	112	19.9 %	9.0 %	12.4 %	6.6 %
B2_K40_MAX2	290	74	25.5 %	5.9 %	9.6 %	5.0 %
B2_K40_MAX3	365	76	20.8 %	6.1 %	9.4 %	4.9 %
B2_K50_MAX2	239	57	23.8 %	4.6 %	7.7 %	4.0 %
B2_K50_MAX3	326	68	20.9 %	5.4 %	8.6 %	4.5 %
B2_K60_MAX3	291	71	24.4 %	5.7 %	9.2 %	4.8 %

TABLEAU 7 – Résultats globaux des expérimentations Llama zero-shot

La meilleure configuration Llama est B2_K40. Elle obtient 112 paires communes avec les experts, contre 80 pour B2_K30, et améliore le F1 de 9.3 % à 12.4 %. Cette configuration montre que l'augmentation du nombre de techniques candidates MITRE ATT&CK peut aider le modèle à retrouver davantage de mappings corrects. En revanche, elle génère aussi davantage de solutions : 562 contre 471 pour B2_K30.

Les variantes avec limitation du nombre de mappings par capacité VERIS répondent à un autre objectif : réduire le nombre total de mappings générés. La configuration

B2_K40_MAX2 descend à 290 solutions et améliore la précision à 25.5 %. Cependant, elle ne retrouve que 74 paires communes, contre 80 pour B2_K30 et 112 pour B2_K40. L'objectif de réduction du nombre de solutions est donc atteint, mais l'objectif d'augmentation du nombre de paires communes ne l'est pas.

Ces résultats montrent que la limitation du nombre de mappings par capacité VERIS permet de réduire les faux positifs, mais qu'elle peut aussi supprimer des mappings corrects. Le meilleur résultat Llama en performance globale reste donc B2_K40, tandis que B2_K40_MAX2 constitue la variante la plus stricte.

5.3.3 Test complémentaire avec Qwen

Afin de vérifier si les résultats obtenus avec Llama pouvaient être améliorés par un autre modèle de langage, des tests complémentaires ont été réalisés avec le modèle Qwen/Qwen2.5-7B-Instruct-Turbo, également accessible via l'API Together. L'objectif n'était pas de modifier l'approche principale, mais de comparer plusieurs LLM généralistes sur la même tâche de génération de mappings VERIS vers MITRE ATT&CK.

Plusieurs configurations ont été évaluées en faisant varier la taille des lots et le nombre de techniques MITRE candidates fournies au modèle. Les résultats obtenus sont présentés dans le tableau 8.

Configuration	Solutions	Communes	Précision	Rappel	F1	Jaccard
Qwen25 7B B1_K30	875	68	7.8 %	5.4 %	6.4 %	3.3 %
Qwen25 7B B2_K40	642	53	8.3 %	4.2 %	5.6 %	2.9 %
Qwen25 7B B1_K40	272	24	8.8 %	1.9 %	3.2 %	1.6 %
Qwen25 7B B1_K60	350	12	3.4 %	1.0 %	1.5 %	0.8 %

TABLEAU 8 – Comparaison des configurations Qwen testées pour le mapping VERIS vers MITRE ATT&CK.

La meilleure configuration Qwen est Qwen25 7B B1_K30. Elle génère 875 paires de mapping, dont 68 sont communes avec la référence experte, pour une précision de 7.8 %, un rappel de 5.4 % et un F1 global de 6.4 %. Cette configuration améliore les autres variantes Qwen testées, notamment B2_K40, qui atteint 53 paires communes et un F1 de 5.6 %.

Cependant, ces résultats restent nettement inférieurs à ceux obtenus avec Llama. La meilleure configuration Llama, LLAMA_B2_K40, produit 562 paires de mapping, dont 112 communes avec les experts, avec une précision de 19.9 %, un rappel de 9.0 % et un F1 de 12.4 %. Llama conserve donc un meilleur équilibre entre volume de mappings générés, nombre de correspondances correctes et qualité globale de la prédiction.

Ces tests montrent que le choix du LLM influence fortement la qualité du mapping généré. Même si Qwen a pu être exécuté correctement, il produit davantage de faux positifs et retrouve moins de correspondances expertes que Llama. Qwen n'est donc pas retenu comme meilleure variante LLM ; il reste un essai complémentaire permettant de confirmer que LLAMA_B2_K40 est la configuration LLM la plus pertinente parmi celles testées.

5.3.4 Synthèse des résultats Fine-tune

Les résultats de la solution Fine-tune montrent que le modèle supervisé local constitue la meilleure variante de cette approche. La configuration FINE_TUNE_T005_K20 obtient

le meilleur équilibre entre précision et rappel, avec un F1 de 41.3 %. Elle retrouve 576 paires communes avec les mappings experts, ce qui indique que l'apprentissage supervisé à partir des associations existantes est particulièrement adapté à cette tâche.

Les variantes LLM zero-shot restent utiles pour l'analyse, mais elles sont moins performantes dans ce contexte. La meilleure configuration Llama est B2_K40, avec un F1 de 12.4 %. Elle améliore la configuration de référence B2_K30, mais reste limitée par l'absence d'entraînement spécifique sur les mappings VERIS-MITRE.

Le test complémentaire avec Qwen confirme cette tendance. Même si le modèle Qwen25 7B B1_K30 a pu générer des mappings exploitables, ses résultats restent inférieurs à ceux de Llama. Dans le cadre de cette solution, le modèle local supervisé reste donc la variante la plus pertinente, tandis que les modèles LLM zero-shot servent surtout de comparaison expérimentale.

5.4 Résultats — Solution PROMPT

5.4.1 Résultat de la premiere run

L'expérimentation du mapping se base sur les versions suivantes : VERIS 1.4.1 et ATT&CK 19.1 via TogetherAI.

Le Tableau 9 résume le **premier run** (pipeline initial, avant correctifs).

Capability	Kimi-K3	DeepSeek-V4-Pro
action.hacking	JSON tronqué	vide
action.malware	JSON tronqué	vide
action.social	JSON tronqué	vide
attribute.availability	complet	vide
attribute.confidentiality	vide	vide
attribute.integrity	vide	vide
value_chain.development	vide	vide

TABLEAU 9 – Premier run PROMPT (avant correctifs de pipeline).

Kimi-K3. :

Seul la capability `attribute.availability` (petite capability) a produit un JSON valide, les autres capabilities (plus larges) ont été coupés en sortie.

DeepSeek-V4-Pro. :

Les sept fichiers vides (timeout et/ou tokens de raisonnement sans `content` exploitable).

5.4.2 Seconde run

Après les correctifs appliquées (voir 4.5.7), une nouvelle expérimentation de mapping a été effectuée avec les mêmes versions des bases de donnée (VERIS 1.4.1 et ATT&CK 19.1). Cette fois, les sept fichiers JSON de capability sont complets et valides pour les deux modèles. Les fichiers sont ensuite enrichis au format attendu par le script de comparaison (`compare_veris_mappings_v2.py`)

Le tableau 10 compare les performances globales agrégées au mapping expert.

Modèle	Paires sol.	Communes	Préc.	Rapp.	F1
Kimi-K3	770	298	38,7 %	23,8 %	29,5 %
DeepSeek-V4-Pro	2231	523	23,4 %	41,8 %	30,0 %

TABLEAU 10 – Second run PROMPT vs mapping expert (agrégé, 1250 paires expertes).

Capability	Exp.	Kimi-K3			DeepSeek F1		
		Prec.	Rapp.	F1	Prec.	Rapp.	F1
action.hacking	534	37,3	25,1	30,0	21,3	30,5	25,1
action.malware	405	40,0	17,3	24,1	21,1	44,4	28,6
action.social	79	31,7	25,3	28,2	31,2	31,6	31,4
attribute.integrity	91	25,3	26,4	25,8	13,4	51,6	21,3
attribute.confidentiality	73	100,0	26,0	41,3	59,0	94,5	72,6
attribute.availability	44	58,8	45,5	51,3	52,7	65,9	58,6
value_chain.development	25	44,0	44,0	44,0	88,9	32,0	47,1

TABLEAU 11 – Second run PROMPT : détail par capability group (%).

Les deux modèles atteignent un F1 global de $\approx 30\%$.

Ils divergent surtout sur le compromis. En effet, on a deux positions :

- Kimi-K3 propose moins de paires avec une précision plus haute ($\approx 38.7\%$)
- DeepSeek-V4-Pro quand à lui propose beaucoup plus de paires avec un rappel supérieur ($\approx 41,8\%$)

On remarque donc que **Kimi-K3** fait un mapping plus sélectif, tandis que **DeepSeek-V4-Pro** fait une couverture plus large, au prix de davantage de faux positifs.

Le gain le plus marqué apparaît sur **attribute.confidentiality** (DeepSeek F1 72,6 %) et **attribute.availability** (F1 $> 50\%$ pour les deux), alors que les grands groupes **hacking/malware** restent les plus difficiles.

6 Discussion

7 Conclusion